

Private Delegation of (Non-)Membership Proof Updates in Cryptographic Accumulators

Bence Soóki-Tóth¹, Botond Glasz², Alireza Kavousi³, and István András Seres²

¹ Aarhus University

² Eötvös Loránd University

³ University College London

Abstract. A universal, dynamic accumulator is a verifiable data structure that compresses a set of elements (e.g., unspent coins, issued public key certificates, etc.) into a succinct digest while supporting addition and deletion of elements alongside efficient proving of (non-)membership in that set. In many applications, valid (non-)membership proofs are a prerequisite to accessing a service (e.g., sending a private payment transaction, establishing a TLS connection, etc.). Typically, newly added or deleted elements necessitate updating all existing (non-)membership proofs per update. Thus, intermittently connected clients will possess invalid (non-)membership proofs whenever they reconnect. In this work, we design, implement, and evaluate algorithms for the RSA and bilinear accumulators that allow a resource-constrained client to privately delegate the updates of its (non-)membership proofs to an untrusted server. We define and prove security in a game-based framework under standard assumptions. We also study proof delegation in the batch setting. The on-line client algorithms are constant-time, i.e., independent of the updated set size k , compared to prior $\mathcal{O}(k)$, $\mathcal{O}(\sqrt{k})$ works. The private delegation algorithms for membership proofs incur an asymptotically constant computational overhead for the server (e.g., $6.12\times$ overhead when 2^{10} elements were added in the offline phase to the RSA accumulator) compared to the non-private membership proof creation algorithms.

Keywords: Cryptographic Accumulator · Proof Delegation · RSA Accumulator · Bilinear Accumulator · Membership Proof

1 Introduction

Imagine a client owning an element x in a set S_t at time t , *e.g.*, unspent coins in a cryptocurrency ledger, public keys in a key directory, etc. Suppose that the set S_t is large, *i.e.*, $|S_t| \gg 2^{10}$, and more importantly, that it is *dynamically changing* over time. In other words, operators can add and remove elements from the set S_t , *i.e.*, typically $S_t \neq S_{t'}$. We assume that S_t is represented succinctly as Acc_t , *i.e.*, an accumulator value using a universal, dynamic cryptographic accumulator scheme [9,52]. Suppose that the client can use a service (*e.g.*, send a cryptocurrency transaction [56], end-to-end encryption), if it can prove with a (non-)membership proof $\pi_{t,x}$ ($\overline{\pi_{t,x}}$) that $x \in S_t$

($x \notin S_t$). We observe that in many applications of cryptographic accumulators (e.g., stateless blockchains [14,24], registration-based encryption [37,40], transparency logs [54]), clients are running on resource-constrained devices, and are typically intermittently connected. Thus, whenever they reconnect after an offline period, they possess an outdated (non-)membership proof disabling them from accessing the desired services. Thus, the clients' goal is to possess a valid (non-)membership proof $\pi_{t,x}$ ($\overline{\pi_{t,x}}$) corresponding to the element x at time t *without forfeiting their privacy, i.e., releasing anything about the element x* . Our goal in this work is fourfold as follows.

Proof Delegation Resource-constrained clients should be able to delegate the updates of their (non-)membership proofs $\pi_{t,x}$ ($\overline{\pi_{t,x}}$) for all time t , element x to an untrusted server, we call *proof-serving node* (PSN).

Privacy The proof-serving node should not learn anything about the element x , it updates the (non-)membership proofs for. More precisely, two (non-)membership updates should be computationally indistinguishable.

Client Efficiency Ideally, the clients' computational complexity in updating their (non-)membership proofs is in $\mathcal{O}(\text{poly}(\log(k)))$, whenever the server inserts/deletes k elements into/from the set S_t , cf. Table 1.

Public Verifiability Serving thousands of users and updating their accumulator proofs is a resource-intensive task. Thus, PSNs should be financially incentivized properly to serve their clients. To that end, we devise efficient, publicly verifiable proofs for (non-)membership proof updates allowing a PSN to prove that they successfully completed their computational tasks.

We only trust the third-party PSNs for availability (*i.e.*, the server is online), but not for privacy or correctness. Note that there are already established efficient markets (e.g., the Golem network [41]) for verifiably outsourcing computational tasks [42].

1.1 The Necessity of Frequent Proof Updates

Recently, a growing body of literature has been developed that exposes the limits of accumulators and related cryptographic primitives, such as vector commitments. Christ and Bonneau [27] showed that there is no interesting asymptotic trade-off between the accumulator's digest size and the number of proofs that must be updated. In other words, either for n elements the accumulator digest is $\mathcal{O}(1)$ -sized and every membership proof must be updated or the accumulator digest is $\mathcal{O}(n)$ -sized (*i.e.*, the entire database is stored), and thus no proofs need to be updated. The authors demonstrate that this impossibility result is universal and applies to all major authenticated data structures used in blockchains, including cryptographic accumulators, vector commitments, and authenticated dictionaries. As a concrete example, adding a single leaf to a standard Merkle tree [55] changes the root and potentially every node on the path, invalidating the witnesses/proofs for all existing leaves. This results in $\mathcal{O}(n^2)$ total witness updates to add n items. In a follow-up work, Bonneau *et al.* [16] show that

when inserting n new elements into a positive accumulator, then each membership proof needs to be updated $\omega(1)$ times. In fact, more is true, *i.e.*, any accumulator with succinct state (polylogarithmic size) must incur at least $\omega(n)$ total witness updates when adding n items. Abusalah *et al.* [1] investigate the same question of update frequency in the setting where *a single element* is added to a positive accumulator. They found that in an accumulator with n elements providing λ -bit security, essentially all proofs, *i.e.*, $n - \text{negl}(\lambda)$ proofs need to be updated when a new element is inserted. These results underscore the inevitable need for PSNs to make accumulators and vector commitments practical.

1.2 Applications

We detail three promising applications for delegating (non-)membership proof updates privately in cryptographic accumulators.

Stateless Blockchains In traditional blockchains such as Bitcoin and Ethereum, users can stay offline for a long time and simply broadcast a transaction when ready. In stateless blockchain designs [14,24,65], users must constantly monitor the network to update their proofs as the blockchain state evolves. If they did not update their proofs would become stale, and they could not spend their coins. Roughly speaking, the state of a cryptocurrency system at time t is essentially the set $S_t = \{x_i\}_{i=1}^n$ consisting of all the unspent coins in the system. Each new transaction removes elements from S_t and inserts new unspent coins into S_t , updating it to a new $S_{t'}$. However, storing the entire set S_t is burdensome for smaller clients.⁴ In a stateless blockchain, the full nodes store the entire set S_t , while clients only store an accumulator digest Acc_t to the S_t . Whenever clients want to spend a coin x , they can do so by issuing a valid membership proof $\pi_{t,x}$ along with other authorization material, *i.e.*, digital signatures, and other proofs. Interestingly, in a privacy-preserving cryptocurrency system, such as Zcash [7], clients issue a zero-knowledge proof-of-knowledge of owning $\pi_{t,x}$, since clients would lose their privacy if they disclose which unspent coins they want to spend. This design motivates the need for *privately* delegating the updates of (non-)membership proofs in the stateless blockchain design context.

Anonymous Credentials with Revocations In anonymous credential systems, cryptographic accumulators are commonly employed to encode the revocation state of credentials in a compact and publicly verifiable manner, enabling users to prove (non-)membership of a hidden revocation handle. However, a practical challenge is that accumulator witnesses must be updated whenever the underlying set changes, which is problematic for intermittently connected users such as mobile wallet holders. This paradigm underlies classical credential frameworks based on Camenisch–Lysyanskaya signatures [20,21] as well as

⁴ For example, the set of unspent coins in Bitcoin takes up tens of gigabytes [31].

pairing-based constructions [13]. Dynamic RSA and bilinear accumulators provide succinct witnesses and scalable revocation [8,57], but require users to update witnesses whenever the accumulator changes. These designs make large-scale real-world credential revocation system deployments⁵ infeasible. Our framework extends this model by allowing users to outsource witness updates to an untrusted server while preserving unlinkability and public verifiability, thereby improving practicality for intermittently connected users in large-scale anonymous credential deployments, for example European Digital Identity wallet.

Registration-based Encryption Registration-Based Encryption [37,38] (RBE) is a cryptographic primitive designed to eliminate the trusted key authority present in Identity-Based Encryption [15] by replacing it with an untrusted “key curator” that maintains a succinct digest of all registered user public keys. This system suffers from the same efficiency bottlenecks as stateless blockchains, as both rely on a dynamic global commitment that changes with every newly registered user. Just as spending a coin in a stateless blockchain invalidates the inclusion proofs for other users, registering a new user in an RBE system updates the global digest, forcing all existing users to download data and update their own decryption keys to remain compatible with the new state. This is information-theoretically impossible to avoid [16], *i.e.*, users cannot simply generate a key once and go offline; instead, they face a witness update burden that scales with the number of new registrations (specifically $\Theta(n \log n / \log \log n)$), mirroring the heavy bandwidth requirements that plague stateless cryptocurrency designs.

1.3 Strawman Solutions

A straightforward idea is to simply send the (non-)membership proof $\pi_{t,x}$ ($\overline{\pi_{t,x}}$) to the PSN to update these proofs as the set S_t dynamically evolves over time. This strawman solution certainly outsources (non-)membership updates, however, at the cost of sacrificing privacy. Note that the PSN can break privacy by brute-forcing the (non-)membership verification checks, *i.e.*, for instance, for membership proofs, the PSN checks $\forall s_i \in S_t : \text{VerMem}(\text{Acc}_t, \pi_{t,x}, s_i) \stackrel{?}{=} 1$, which holds iff. $s_i = x$. Assuming $|S_t| = \text{poly}(\lambda)$ in the security parameter λ , this brute-force computation can be carried out efficiently.

To preserve the privacy of the outsourced element x , one might apply fully homomorphic encryption on the outsourced (non-)membership proofs [39]. In detail, the client could homomorphically encrypt the accumulator proofs $\pi_{t,x}$ ($\overline{\pi_{t,x}}$) and send the ciphertexts to the PSN, who completes the (non-)membership proof update on the homomorphic ciphertext. This approach solves the problem of privately delegating accumulator proofs, but yields a less practical solution than applying our tailor-made techniques.

⁵ European Digital Identity wallet [34] must support ≈ 450 million users by late 2026.

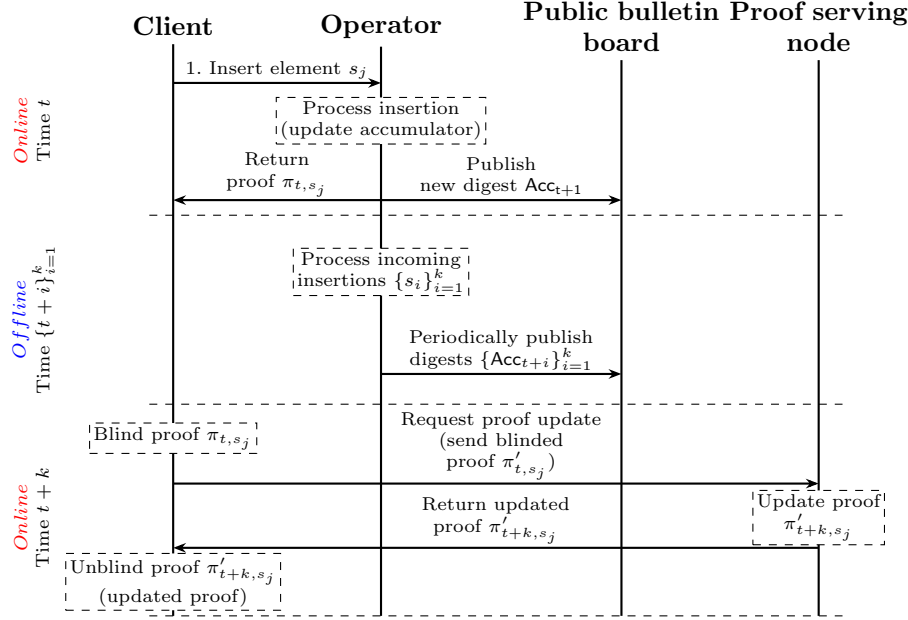


Fig. 1: A sequence diagram illustrating the interactions of participants (*cf.* Section 4) in a privately delegatable cryptographic accumulator scheme. In a private delegation protocol for (non-)membership proof updates, at time $t+k$, the client requests a proof serving node the update of an outdated proof π_{t,s_j} for a blinded element s_j to the latest valid proof π_{t+k,s_j} .

1.4 Technical Overview

In this work, our goal is to endow universal, dynamic accumulators with efficient algorithms that allow a client and a server—also known as proof serving node (PSN)—to update the client’s (non-)membership proofs privately, *cf.* Figure 1.

Syntax and Security Requirements We introduce four efficient PPT algorithms `BlindProof`, `BlindProofUpd`, `VerBlindProofUpd`, and `UnBlindProof`, to allow private delegation of (non-)membership proof updates in accumulators.

The `BlindProof`($\cdot, \cdot, \pi_{t,x}$) $\rightarrow \pi'_{t,x}$ algorithm is run by the client, and blinds the (non-)membership proof $\pi_{t,x}$ to $\pi'_{t,x}$. Given a blinded (non-)membership proof $\pi'_{t,x}$ the server runs `BlindProofUpd`($\text{Acc}_t, \text{Acc}_{t'}, S_{t'}, \pi'_{t,x}$) $\rightarrow (\pi'_{t',x}, \text{aux})$ to update the proof with respect to the current accumulator digest $\text{Acc}_{t'}$ and produces a succinct proof of update correctness aux . Public verifiability of the update correctness is checked by the `VerBlindProofUpd`($\text{Acc}_t, \text{Acc}_{t'}, \pi'_{t,x}, \pi'_{t',x}, \text{aux}$) $\rightarrow \{0, 1\}$ algorithm. After the private delegation `UnBlindProof`($\cdot, \cdot, \pi'_{t',x}, \cdot$) $\rightarrow \pi_{t',x}$ is run by the client to unblind $\pi'_{t',x}$ and retain the updated proof $\pi_{t',x}$.

Intuitively, **correctness** for private accumulator proof delegation entails that an honestly run private delegation should always yield a valid (non-)membership proof with respect to the current accumulator value. We formalize our privacy requirement for private accumulator proof delegation protocols in a notion we call **blinded membership proof indistinguishability**. We require that the PSN server when seeing a blinded membership proof $\pi'_{t,x}$ should not be able to tell for which element x the blinded accumulator proof belongs to. We formalize this notion in Definition 5 as a classical indistinguishability game. Finally, we demand **computational delegation soundness** from a private proof delegation protocol. Specifically, a malicious PSN server should not be able to produce a publicly verified update proof aux such that the unblinded (non-)membership proof $\pi_{t',x}$ is rejected by the accumulator scheme's (Non)MemVer($\cdot$) algorithm.

We present constructions for both the bilinear and the RSA accumulator schemes, which yield client-efficient proof updates (that is $\mathcal{O}(1)$, *i.e.*, independent of the update size). We first provide a technical overview of private delegation in the RSA accumulator.

Private Delegation of RSA Accumulator Proofs

Membership proofs In the RSA accumulator for an element x , and set S_t , the membership proof is $\pi_{t,x} = g^{\prod_{s_j \in S_t, s_j \neq x} s_j}$, *i.e.*, it is an x th root of the accumulator digest Acc_t . Let $S_{t'} := S_t \cup \{x_i\}_{i=1}^k$ and $\Delta := \prod_{i=1}^k x_i$. We define the blinded RSA membership proof simply as $\pi'_{t,x} = \pi_{t,x} \cdot g^r$ for an element $r \in \mathbb{Z}_\lambda$ generated by the client uniformly at random. Note that $\pi'_{t,x}$ statistically hides $\pi_{t,x}$. Thus, the client can safely send $\pi'_{t,x}$ to the PSN server. Afterwards, the PSN updates the blinded membership proof $\pi'_{t,x}$ from time t to t' by computing $\pi'_{t',x} := ((\pi'_{t,x})^\Delta, g^\Delta)$. The server also computes discrete logarithm equality proofs aux to show consistency between the public accumulator digests $(\text{Acc}_t, \text{Acc}_{t'} = \text{Acc}_t^\Delta)$ and the (updated) blinded membership proofs $(\pi'_{t,x}, (\pi'_{t,x})^\Delta)$. The server sends $(\pi'_{t',x}, \text{aux})$ to the client. Finally, the client can obtain its updated membership proof $\pi_{t',x}$ by computing $\pi_{t',x} := (\pi'_{t',x})^\Delta / (g^\Delta)^r$ using the randomness r it generated at the beginning.

Non-membership proofs Recall, if $x \notin S_t$, then $\gcd(s^*, x) = 1 (\Leftrightarrow \exists(a, b) \in \mathbb{Z}^2 : as^* + bx = 1)$, where $s^* = \prod_{s_j \in S_t} s_j$. The RSA non-membership proof proves the co-primality of s^* and x in the exponent. More precisely, the non-membership proof in the RSA accumulator is $\overline{\pi_{t,x}} := (a, g^b)$ for $(a, b) \leftarrow \text{Bezout}(s^*, x)$.

One may be tempted to design a protocol that updates the Bezout-coefficients corresponding to x . Assume $S_{t+1} = S_t \cup s'$. We can compute the updated Bezout-coefficients $(a', b') \leftarrow \text{Bezout}(s^* \cdot s', x)$ as a function of (a, b, s^*, s') as follows.

$$a' := au \wedge b' := \frac{1 - aus^*s'}{x}, \quad (1.1)$$

where $u = s' \bmod x$. Computing the expressions in Equation (1.1) in a privacy-preserving manner turns out to be extremely difficult over $\mathbb{Z}$. Thus, we take a

different and simpler approach. The high-level idea is that the client uses the server as an untrusted Bézout-oracle. More precisely, first the client masks the non-element x (instead of the non-membership proof) by computing $\overline{\pi_{t,x}'} := x \cdot q$, which hides x for a random prime q assuming the hardness of factoring and that the primes are chosen from a large enough set. Afterwards, the client asks the server to compute $\overline{\pi_{t',x}'} := (a, g^b) = (a, B)$ for $(a, b) \leftarrow \text{Bezout}(s^*, \overline{\pi_{t,x}'})$. Finally, the client obtains the final non-membership proof as $\overline{\pi_{t',x}} := (a, B^q)$. We note that this construction only satisfies a weaker notion of privacy, cf. Definition 6.

Private Delegation of Bilinear Accumulator Proofs

Membership proofs In the bilinear accumulator, the accumulated set S_t is encoded as the accumulator polynomial $p_t(X) = \prod_{s_i \in S_t} (X - s_i)$, and the corresponding accumulator digest is defined as the KZG polynomial commitment [48] of $p_t(X)$, i.e., $\text{Acc}_t := \text{KZG.Com}(p_t(X))$. The membership proof for an element s_j is the KZG commitment $\pi_{t,s_j} = \text{KZG.Com}(q_{t,s_j}(X))$ to the quotient polynomial $q_{t,s_j}(X) = p_t(X)/(X - s_j)$. Papamanthou observes [58] that when a single element is added to the set $S_{t+1} = S_t \cup s_{n+1}$, then one can simply update the bilinear accumulator membership proofs as

$$\pi_{t+1,s_j} = \text{Acc}_t \cdot \pi_{t,s_j}^{s_{n+1} - s_j} . \quad (1.2)$$

Treating newly added elements as unknown variables, it is straightforward to see that Equation (1.2) suggests when k new elements $\{s_{n+i}\}_{i=1}^k$ are added to S_t , i.e., $S_{t'} = S_t \cup \{s_{n+i}\}_{i=1}^k$, then we must compute a degree- k polynomial function $q^*(X)$ of the quotient polynomial $q(X)$ committed in π_{t,s_j} .

Motivated by the above technique, we define the blinded bilinear membership proof as $\pi'_{t,x} = \{\text{KZG.Com}(\text{crs}, rX^i q(X))\}_{i=0}^k$, where r is a uniformly random element in $\mathbb{F}_p$ and k is the maximum supported number of added elements during the offline period. Intuitively, π'_{t,s_j} is a randomized common reference string encoding $q(X)$ that the server can use to commit to $q(X)q^*(X)$ for any update polynomial $q^*(X)$ of degree $\leq k$, without ever learning $q(X)$.

Non-membership proofs If $s_j \notin S_t$, then $p_t(s_j) = y_{t,s_j} \Leftrightarrow \exists q_{t,s_j}(X) : p_t(X) = q_{t,s_j}(X)(X - s_j) + y_{t,s_j}$, where $y_{t,s_j} \neq 0$. Thus, $\overline{\pi_{t,s_j}} := (\text{KZG.Com}(q_{t,s_j}(X)), y_{t,s_j})$. Observe that after adding a single element s_{n+1} to the set S_t , we obtain the following recurrence relation for the bilinear non-membership proof elements.

$$\begin{aligned} q_{t+1,s_j}(X) &= q_{t,s_j}(X)(X - s_{n+1}) + y_{t,s_j} , \\ y_{t+1,s_j} &= y_{t,s_j} s_j - y_{t,s_j} s_{n+1} . \end{aligned}$$

In Section 5.4, we show how these recurrence relations admit a simple and *interactive* non-membership proof delegation protocol.

1.5 Our Contributions

In this work, we provide the following three contributions.

Scheme	Accumulator	Update msg. size	Client cost	Server cost	Mem/Non-mem	Batching	Decentralized
[18]	Merkle tree [55]	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(k \log_2(S_t))$	✓/✗	✗	✓
[4]	RSA [8]	$\mathcal{O}(k)$	$\mathcal{O}(k)$	$\mathcal{O}(k)$	✓/✗	✗	✓
Section 5.3	RSA [8]	$\mathcal{O}(1) \setminus \mathcal{O}(1)$	$\mathcal{O}(1) \setminus \mathcal{O}(1)$	$\mathcal{O}(k) \setminus \mathcal{O}(k)$	✓/✓	✓	+
[68]	Bilinear [57]	$\mathcal{O}(k)$	$\mathcal{O}(k)$	$\mathcal{O}(k \log_2(k))$	✓/✓	✓	✗
[45]	Bilinear [57]	$\mathcal{O}(\sqrt{k})$	$\mathcal{O}(\sqrt{k})$	$\mathcal{O}(k)$	✓/✗	✗	✓
Section 5.4	Bilinear [57]	$\mathcal{O}(1) \setminus \mathcal{O}(k) \setminus \mathcal{O}(k)$	$\mathcal{O}(1) \setminus \mathcal{O}(k) \setminus \mathcal{O}(k)$	$\mathcal{O}(k) \setminus \mathcal{O}(k)$	✓/✓	✓	✓

Table 1: Comparing related work on delegating privately (non-)membership proofs in accumulators. The column “Mem/Non-mem” indicates that the delegation schemes support (non-)membership proofs. The column “decentralized” denotes whether the delegation schemes assume that an accumulator manager knows the trapdoor of the accumulator (*e.g.*, group order, *crs* trapdoor, etc.) or not. Offline/preprocessing costs are written in parentheses. If a scheme supports both (non-)membership proof delegations, then these protocols’ asymptotics is separated by a backslash. + means that the property can only be achieved under an append-only instantiation, *i.e.*, one that does not support deletion.

Private accumulator proof delegation framework To the best of our knowledge, for the first time, we define private accumulator proof delegation and its security, privacy, and soundness in a game-based security framework.

Efficient constructions We construct the first efficient proof delegation protocols for the RSA [9], and bilinear accumulators [57] that achieve privacy, public verifiability, and constant client complexity.

Open-source implementation We provide the first open-source implementation of private accumulator proof delegation and evaluate its performance. Our code is publicly available at <https://github.com/GlaszBoti/private-accumulator-proof-delegation>.

Organization This paper is organized as follows. In Section 2, we review the related work. We provide the necessary pertinent background in Section 3. We describe our system model in Section 4. We introduce our constructions in Section 5, and optimizations in Section 6. We discuss our implementation and performance evaluation in Section 7. Finally, we conclude our paper in Section 8 with open problems and future directions.

2 Related Work

2.1 Accumulators and Privacy

The anonymous credential literature [20,22] often relies on cryptographic accumulators to store revoked credentials [19]. These works typically do not provide privacy for the witness update operation [52,19,62], *i.e.*, the server knows which witness it is updating. Even if they provide privacy-preserving witness update operations, they do not offer poly-logarithmic update costs for clients [4,45] *cf.* Table 1. To the best of our knowledge, there are two private membership proof

delegation schemes [68,18] that provide a constant-time client, a necessity in real-world applications. The delegation scheme of [68] works in the accumulator manager setting for the bilinear accumulator [68], *i.e.*, incompatible with decentralized applications. The scheme of [18] for Merkle trees applies heavy recursive SNARK machinery, we wish to concretely outperform. Baldimtsi *et al.* create new accumulators that hide the elements and set size, *i.e.*, a so-called oblivious accumulator [5]. In the applications we have in mind, the accumulated set is typically public. Barthoulot *et al.* explore delegating accumulator proofs privately to a server in a generic setting [6]. They characterize what properties a generic solution must satisfy, and they conclude that likely such a construction does not exist. In contrast, we show non-generic techniques that are tailored to the accumulator schemes at hand in a white-box manner. Thus, we can circumvent their impossibility result. Bowe and Miers devised updatable membership proofs for Merkle trees using SNARK recursion in the context of privacy-preserving cryptocurrencies [18]. However, they provided neither security proofs for their constructions nor implementation and performance evaluation. In contrast, in this work, we do not resort to heavy techniques such as proof recursion.

2.2 Delegation of Cryptographic Computations

The study of securely delegating cryptographic computations to a malicious server was initiated by Hohenberger and Lysyanskaya [44]. In particular, to our discussion, outsourcing (modular) exponentiations is the most related research direction [70,25,61]. These works mostly study the problem of outsourcing an exponentiation $g^u \in \mathbb{G}$, where both the base g and the exponent u must be kept secret from the helping server. Additionally, the server wants to ensure the verifiability of the output g^u . At a first glance, one may think that updating (non-)membership proofs in certain accumulators (e.g., the RSA accumulator) is “just” outsourcing an exponentiation to an untrusted server. Observe that in our setting, these techniques are not applicable, since the client does not even know the exponent u (derived from the newly added/deleted items) it wishes to raise its outdated membership proof g . An added challenge we must solve is to show that the output of the outsourced cryptographic computation is correct w.r.t. the latest accumulator digest. The typical output verifiability notion considered in the aforementioned papers is insufficient in our applications, as the server must show validity to a public value, *i.e.*, the accumulator’s latest digest.

2.3 Private Information Retrieval

The problem of privately delegating (non-)membership proofs for cryptographic accumulators can be cast [45] as a Private Information Retrieval (PIR) problem [26]. Given a database of valid membership proofs $D_t = \{\pi_{t,s_j}\}_{s_j \in S}$, the client wants to obtain $D_t[i] := \pi_{t,s_i}$ for some index $i \in \mathcal{I}$ without disclosing the index i to the server [45]. An added challenge in our setting is that the database is dynamically changing; most PIR protocols do not support this efficiently [53]. There are a multitude of lower bounds for single-server PIR protocols, *e.g.*, a

Notation	Description
x, s_j	Element in the accumulated set
S_t	The accumulated set S at time t
n	The size of S_t , <i>i.e.</i> , $ S_t := n$
k	The number of added elements from t to t'
l	The number of deleted elements from t to t'
Acc_t	The accumulator digest of the set S at time t
$\pi_{t,s_j}(\overline{\pi_{t,s_j}})$	(Non-)membership proof for s_j at time t
$\text{Primes}(\lambda)$	The first $2^{2\lambda}$ prime numbers in $\mathbb{N}$
$\text{H}_{\text{prime}}(\cdot)$	A hash function from $\{0,1\}^*$ to $\text{Primes}(\lambda)$

Table 2: Notations used throughout the paper.

typical lower bound for PIR protocols in the information-theoretic setting states that $\Omega(n)$ bits must be communicated by the server, where n is the database size [43]. In the preprocessing setting, there is a tradeoff of online and offline communication complexity. In particular, Corrigan-Gibbs and Kogan showed the lower bound for any preprocessing PIR, *i.e.*, $C \cdot T \geq \Omega(n)$, where C denotes the bits of communication in the offline phase, and T denotes the number of bits probed of the database in the online phase [29]. In contrast, we create protocols where the client's communication and computation complexity are both $\mathcal{O}(1)$ by exploiting the algebraic structure of the underlying database $D_t = \{\pi_{t,s_j}\}_{s_j \in S}$.

3 Background

3.1 Notations

The randomization of a group/field element x is denoted as x' . We use a multiplicative notation for the group operation in both the elliptic curve and the group of unknown order contexts. Deterministic assignment is denoted as $=$, while probabilistic assignment or uniform random sampling from a set S is denoted as $x \leftarrow S$. Our notation is summarized in Table 2.

3.2 Universal, Dynamic Accumulator Syntax

A universal, dynamic accumulator Π is a tuple of six probabilistic polynomial time (PPT) algorithms [8, 5] as follows.

- Setup**(1^λ) $\rightarrow$ **pp**. Given a security parameter λ , this algorithm returns the public parameters **pp**. The **pp** is implicitly given to all of the following algorithms.
- Add**(Acc_t, x) $\rightarrow$ (Acc_{t+1}, u). Given an accumulator Acc_t and element x , the addition algorithm returns the new accumulator Acc_{t+1} with x added to the set of accumulated values and an update information u .
- Del**(Acc_t, x) $\rightarrow$ (Acc_{t+1}, u). Given an accumulator Acc_t and element x , the deletion algorithm returns the new accumulator Acc_{t+1} with x being deleted from the set of accumulated values S_t and an update information u .

- (Non)MemProofCreate(S_t, Acc_t, x) $\rightarrow \pi_{t,x}$. Given a set S_t at time t , an accumulator digest Acc_t , and element x , the (non-)membership creation algorithm returns a (non-)membership proof $\pi_{t,x}$.
- (Non)MemProofUpdate($\text{Acc}_t, \pi_{t,x}, S_{t'}$) $\rightarrow \pi_{t',x}$. Given an old accumulator value Acc_t , an outdated membership proof $\pi_{t,x}$, and the latest set $S_{t'}$, the proof update algorithm returns an updated (non-)membership proof $\pi_{t',x}$.
- (Non)MemVer($\text{Acc}_t, x, \pi_{t,x}$) $\rightarrow \{0, 1\}$. For a digest Acc_t , an element x , and a (non-)membership proof $\pi_{t,x}(\overline{\pi_{t,x}})$, the (non-)membership verification algorithm outputs 1 or 0 indicating accept or reject.

Definition 1 (Correctness). *An accumulator scheme Π is correct if all honestly generated proofs are accepted with overwhelming probability. More formally, $\forall x \in S_t$:*

$$\Pr(\text{MemVer}(\text{Acc}_t, x, \text{MemProofCreate}(\text{Acc}_t, x)) = 1) \geq 1 - \text{negl}(\lambda) .$$

Definition 2 (Soundness). *An accumulator scheme Π is sound if all adversarially generated proofs are accepted with negligible probability. More formally, $\forall x \notin S_t$:*

$$\Pr(\text{MemVer}(\text{Acc}_t, x, \text{MemProofCreate}(\text{Acc}_t, x)) = 1) \leq \text{negl}(\lambda) .$$

3.3 The RSA and Bilinear Accumulators

We recall the efficient algorithms of the RSA [8,52] and bilinear [57,30] universal dynamic accumulators in Figures 11 and 12.

3.4 Non-interactive Zero-knowledge Proofs

We recall the relevant syntax of non-interactive zero-knowledge (NIZK) proofs following [11], and for the details and exact security requirements, we refer to [11]. NIZK arguments consist of four PPT algorithms that are defined with respect to a relation generator algorithm $\mathcal{R}\text{-Gen}(1^\lambda)$ that, upon receiving some security parameter λ , outputs a polynomial-time decidable relation $\mathcal{R} : \{0, 1\}^* \times \{0, 1\}^*$ for which $(\phi, w) \in \mathcal{R}$, where ϕ is typically an algebraic statement in a ring $\mathbb{F}_N$ or in a finite field $\mathbb{F}_p$ and w is a valid witness for the instance.

- $\text{NIZK.Setup}(\mathcal{R}) \rightarrow (\text{crs}, \tau)$. For the relation $\mathcal{R}$, the setup produces a common reference string crs and a simulation trapdoor τ . This possibly randomized algorithm may use public/private coins (a.k.a. transparent/trusted setup).
- $\text{NIZK.Prove}(\mathcal{R}, \text{crs}, \phi, w) \rightarrow \pi$. Given $(\phi, w) \in \mathcal{R}$ and the common reference string crs , the prover returns an argument π .
- $\text{NIZK.Verify}(\mathcal{R}, \text{crs}, \phi, \pi) \rightarrow \{0, 1\}$. the relation $\mathcal{R}$, the common reference string crs , the statement ϕ , and an argument π , the verification algorithm returns 0 or 1 indicating reject or accept.
- $\text{NIZK.Sim}(\mathcal{R}, \tau, \phi) \rightarrow \pi$. Using the simulation trapdoor τ of the relation $\mathcal{R}$, and the statement ϕ , the simulator returns a simulated proof π .

This work builds on NIZKs for the following efficiently decidable languages in both groups of unknown order and elliptic-curve groups. All of the corresponding proofs have constant size and constant-time verification.

Discrete Logarithm equality (DLEq) [23,14].

$$\mathcal{R}_{\text{DLEq}} = \{(((g, u, h, v) \in \mathbb{G}^4); w \in \mathbb{Z}) : u = g^w \wedge v = h^w\}. \quad (3.1)$$

Polynomial Exponentiation Equality (PoEEq) A variant of the DLEq proof for “exponentiating” with a polynomial, *cf.* [66].

$$\mathcal{R}_{\text{PoEEq}} = \{(((g, u, h, v) \in \mathbb{G}^4, \phi(X) \in \mathbb{F}_p[X]); \tau \in \mathbb{F}_p) : u = g^{\phi(\tau)} \wedge v = h^{\phi(\tau)}\}. \quad (3.2)$$

4 System and Threat Models

In this work, we apply a discrete-time model. A timer $t \in \mathbb{N}$ is only incremented when an item is inserted or deleted from the accumulator with digest Acc_t , *i.e.*, we update our timer from t to $t+1$, and the digest to Acc_{t+1} . Next, we describe our system model illustrated in Figure 2.

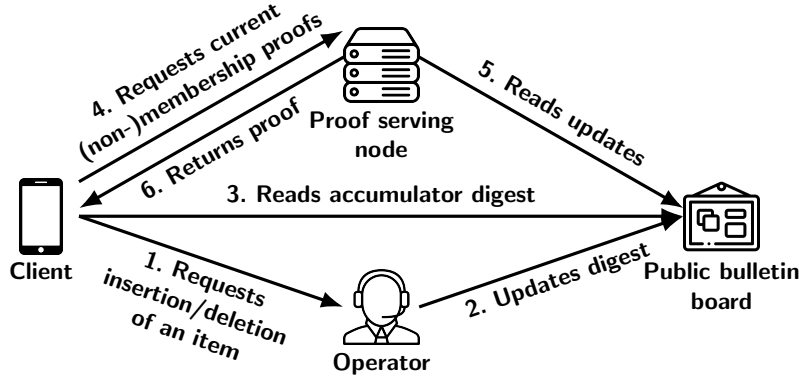


Fig. 2: Our System Model. ① A client requests the insertion/deletion of an item into/from the accumulated set. ② Subsequently, the operator regularly updates the accumulator digest and publishes it to a public bulletin board (*e.g.*, a blockchain) to reflect its current state. ③ The client (and all other parties as well) can read the latest accumulator digest and elements from the bulletin board. ④ After an offline period, the client may request its latest valid (non-)membership proofs from a proof serving node (PSN). ⑤–⑥ Finally, the PSN serves the client with its latest (blinded) valid (non-)membership proofs.

Client We assume resource-constrained, intermittently connected clients, *e.g.*, a browser, a cryptocurrency wallet, smart watch, etc., that want to privately delegate the updates of (non-)membership proofs for an element $x \in S$ ($x \notin S$). The client possesses a valid (non-)membership proof for an item x w.r.t Acc_t , and wishes to obtain a valid (non-)membership proof for $\text{Acc}_{t'}$, where $t' > t$. Naturally, the client does not know which elements were added or removed from the set S between time t and t' .

Public Bulletin Board There is a trusted public bulletin board (*e.g.*, a blockchain or an independent cloud server) readable by everyone, acting as a source of truth, *i.e.*, it stores the current accumulator digest Acc_t at all times t , enabling clients to verify (non-)membership proofs.

Operator The public bulletin board is operated by a set of incentivized parties. For instance, they are validators (*i.e.*, stakers of proof-of-work miners) in the blockchain context or certificate authorities in the key transparency application. The operator appends new elements, removes necessary elements from the accumulator, updates the accumulator digest on the public bulletin board *i.e.*, manages the accumulator.

Proof Serving Nodes The so-called proof serving nodes (PSNs) are untrusted third parties that are always online and, upon request, privately update the clients' (non-)membership accumulator proofs. In practical accumulator deployments, PSNs and operators may be the same entities, but for conceptual reasons, we separate them in our discussion.

4.1 Supporting Various Set Operations

Clients can update the accumulated set S_t in various ways depending on the application. We focus mainly on supporting the following two set operations.

Addition Users or the accumulator manager may add new elements to the accumulator *i.e.*, $S_{t+1} := S_t \cup s_{n+1}$. In our target applications, this operation models the creation of new coins in stateless blockchains, or the issuance of new public key certificates in key transparency logs.

Deletion Users or the accumulator manager may remove elements from the set, *i.e.*, $S_{t+1} := S_t \setminus s_j$ for some $s_j \in S_t$. This operation models the spending of a cryptocurrency coin, or the revocation of a public key certificate.

We admit that there are more interesting set operations that we may consider, *e.g.*, element modification or element modification by some allowed function $f(\cdot)$, *i.e.*, $s_{t+1,j} := f(s_{t,j})$. We leave it to future work to devise privately delegatable accumulator schemes supporting other set operations as well.

4.2 Threat Model

As we show later, our protocols offer strong privacy guarantees, *i.e.*, the server cannot distinguish two (non-)membership proof updates for the same time interval (*cf.* Definition 5). However, we acknowledge that the PSN can use various

side-channel information to breach user privacy. For example, the PSN server may take advantage of update request timing correlations, repeated queries, or other correlation attacks. We consider these leakages to be orthogonal to this work, as they can be circumvented by the appropriate use of anonymous communication tools, *e.g.*, Tor, virtual private networks, etc.

5 Privately Delegating Accumulator Proofs

We aim to endow any universal, dynamic accumulator scheme with four new efficient algorithms.⁶

5.1 Syntax of Privately Delegatable Cryptographic Accumulators

We introduce privately delegatable cryptographic accumulator schemes, *i.e.*, we endow universal, dynamic accumulator schemes $\mathcal{ACC} = (\text{Setup}, \text{Add}, \text{Del}, \text{MemProofCreate}, \text{MemVer}, \text{NonMemProofCreate}, \text{NonMemVer})$ with four new algorithms *i.e.*, $\Pi_{\mathcal{ACC}} = (\text{BlindProof}, \text{BlindProofUpd}, \text{VerBlindProofUpd}, \text{UnBlindProof})$ enabling to privately delegate membership proof updates in $\mathcal{ACC}$. For the sake of notational simplicity, we define our syntax and security definitions only for privately delegating *membership proofs*. It is straightforward to define them analogously for non-membership proofs as well.

Definition 3 (Privately Delegatable Accumulator). *We say an accumulator scheme $\mathcal{ACC}$ has privately delegatable membership proof updates if, in addition, it also implements the following four PPT algorithms $\Pi_{\mathcal{ACC}} = (\text{BlindProof}, \text{BlindProofUpd}, \text{VerBlindProofUpd}, \text{UnBlindProof})$ where the public parameters pp of $\mathcal{ACC}$ are assumed to be implicitly given.*

$\text{BlindProof}(\text{Acc}_t, x, \pi_{t,x}) \rightarrow (\pi'_{t,x}, \text{st})$. *Given a membership proof $\pi_{t,x}$ for the element x at time t , this algorithm returns a blinded membership proof $\pi'_{t,x}$ and a state st .*

$\text{BlindProofUpd}(\text{Acc}_t, \{x_i\}_{i=1}^k, \{y_i\}_{i=1}^l, \text{Acc}_{t'}, \pi'_{t,x}) \rightarrow (\pi'_{t',x}, \text{aux})$. *For a blinded membership proof $\pi'_{t,x}$, element x and the old accumulator value Acc_t , the server can update the blinded membership proof $\pi'_{t,x}$ to a new blinded membership proof $\pi'_{t',x}$ so that it will be valid for the updated accumulator value $\text{Acc}_{t'}$ which corresponds to the digest of the set $S_{t'} := (S_t \cup \{x_i\}_{i=1}^k) \setminus \{y_i\}_{i=1}^l$.*

$\text{VerBlindProofUpd}(\text{Acc}_t, \text{Acc}_{t'}, \pi'_{t,x}, (\pi'_{t',x}, \text{aux})) \rightarrow \{0, 1\}$. *For the digests $\text{Acc}_t, \text{Acc}_{t'}$ it is publicly verifiable that the proof update from $\pi'_{t,x}$ to $(\pi'_{t',x}, \text{aux})$ was computed correctly. This deterministic algorithm returns a bit indicating whether the proof delegation was done correctly.*

$\text{UnBlindProof}(\text{Acc}_{t'}, x, \pi'_{t',x}, \text{st}) \rightarrow \pi_{t',x}$. *Given the latest accumulator digest $\text{Acc}_{t'}$, a blinded membership proof $\pi'_{t',x}$ for the element x and a state st , this algorithm returns an unblinded membership proof $\pi_{t',x}$ for $x \in S_{t'}$.*

⁶ Four algorithms for membership proofs, and four for non-membership proofs.

Remark 1. We must blind the membership proof $\pi_{t,x}$ to $\pi'_{t,x}$, otherwise the PSN could link the membership proof to an item x by repeatedly calling the membership verification algorithm of the accumulator scheme. Our model is comprehensive as we allow additions and deletions in the offline period to the accumulator, cf. the `BlindProofUpd( $\cdot$ )` function. Observe that the `VerBlindProofUpd( $\cdot$ )` algorithm does not have private inputs, making the witness update publicly verifiable. We omit $\mathcal{ACC}$ from the subscript of Π as it is obvious from the context which accumulator scheme is endowed with the private delegation algorithms.

5.2 Security and Privacy Requirements

We define the security and privacy requirements for privately delegatable accumulator schemes in a game-based framework. Intuitively, the delegated proof updates when honestly computed are accepted with overwhelming probability.

Definition 4 (Correctness). *Honestly run proof updates yield valid membership proofs with overwhelming probability, i.e., $\forall t, t' \in \mathbb{N}, \forall \text{Acc}_t, \text{Acc}_{t'}, \forall x \in S$, we have*

$$\Pr[\text{VerMem}(\text{Acc}_{t'}, x, \text{UnBlindProof}(\text{Acc}_t, \text{Acc}_{t'}, \pi_{t,x}, (\pi'_{t,x}, \text{aux}))) = 1] \geq 1 - \text{negl}(\lambda) ,$$

where $\pi'_{t,x} \leftarrow \text{BlindProof}(\text{Acc}_t, x, \pi_{t,x})$, and

$$(\pi'_{t',x}, \text{aux}) \leftarrow \text{BlindProofUpd}(\text{Acc}_t, \{x_i\}_{i=1}^k, \{y_i\}_{i=1}^l, \text{Acc}_{t'}, \pi'_{t,x}).$$

Intuitively, the blinded (non-)membership proofs should not leak anything about neither the element x nor the unblinded (non-)membership proof.

Definition 5 (Blinded Membership Proof Indistinguishability). *A privately delegatable accumulator scheme Π has blinded membership proof indistinguishability if $\forall \lambda, \forall$ PPT adversary $\mathcal{A}$, $\exists \text{negl}(\cdot)$ s.t. the advantage of the adversary in the indistinguishability game (cf. Figure 3) is $\text{negl}(\lambda)$. Formally,*

$$\Pr[\mathcal{G}_{\mathcal{A}, \Pi}(\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda) . \quad (5.1)$$

We also introduce a relaxed notion of blinded membership proof indistinguishability. In this weaker privacy notion, the adversary cannot link the blinded membership proof among *exponentially many* accumulated elements with non-negligible probability.

Definition 6 (Weak Blinded Membership Proof Indistinguishability).

Let $x \in_R \{0, 1\}^\lambda$ and $\pi_{t,x}$ a valid membership proof x at time t . Moreover, we let $\pi'_{t,x} := \text{BlindProof}(\text{pp}, \pi_{t,x})$. Then, we say that a privately delegatable accumulator scheme Π satisfies weak blinded membership proof indistinguishability if $\forall (\lambda, t) \in \mathbb{N}^2, x \in_R \{0, 1\}^\lambda$, and $\forall \mathcal{A}$ PPT, there exists a $\text{negl}(\cdot)$ function such that the following holds:

$$\Pr[\mathcal{A}(\text{pp}, \pi'_{t,x}) = x] \leq \text{negl}(\lambda) . \quad (5.2)$$

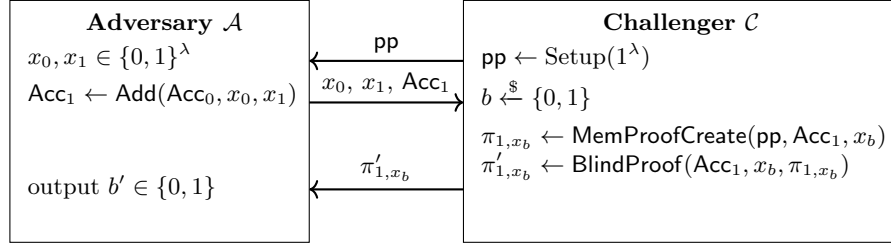


Fig. 3: The blinded membership proof indistinguishability game, cf. Definition 5. The adversary $\mathcal{A}$ wins the game iff. $b = b'$.

In Section 5.3, we will see a private delegation protocol that satisfies weak blinded membership proof indistinguishability, but it does not satisfy the stronger notion of blinded membership proof indistinguishability.

Next, we introduce the notion of delegation soundness. Intuitively, a malicious PSN should not be able to produce a convincing (publicly verifiable) proof update aux for a valid (non-)membership proof $\pi_{t,x}$ such that later the unblinded (non-)membership proof is *not* verified by the accumulators (non-)membership verification algorithms. We define the notion for membership proofs as follows.

Definition 7 ((Computational) Delegation Soundness). A privately delegatable accumulator scheme Π_{ACC} is computationally delegation sound if $\forall \lambda, \forall \text{PPT adversary } \mathcal{A}, \forall x \in S_t, \forall (t, t') \in \mathbb{N}^2 \text{ s.t. } t \leq t', \forall \pi_{t,x} : \text{MemVer}(\text{Acc}_t, \pi_{t,x}, x) = 1$, there exists a $\text{negl}(\cdot)$ function s.t.,

$$\Pr \left[\text{MemVer}(\text{UnBlindProof}(\text{Acc}_{t'}, \text{st}, \pi'_{t',x})) = 0 \mid \text{VerBlindProofUpd}(\text{pp}, \text{Acc}_t, \text{Acc}_{t'}, \pi'_{t,x}, \pi'_{t',x}, \text{aux}) = 1 \right] \leq \text{negl}(\lambda).$$

Definition 8 (Private Delegation of Membership Proof Updates). A universal, dynamic accumulator scheme ACC has a secure, private delegation scheme for membership proof updates if $\Pi_{\text{ACC}} = (\text{BlindProof}, \text{BlindProofUpd}, \text{VerBlindProofUpd}, \text{UnBlindProof})$ satisfies correctness, delegation soundness, and (weak) blinded membership proof indistinguishability.

5.3 Privately Delegating Proofs in the RSA Accumulator

Membership proofs Due to the RSA accumulator's multiplicative structure, a simple $\pi'_{t,x} = \pi_{t,x} \cdot g^r$ randomization suffices for the blinded membership proof, while enabling efficient blinded proof updates. The server computes $\Delta = \prod_{i=1}^k x_i \in \mathbb{Z}$ and the exponentiation $\pi'_{t',x} := (\pi'_{t,x})^\Delta$ to obtain the updated blind membership proof. Finally, the server shows consistency between the accumulator digests $(\text{Acc}_t, \text{Acc}_{t'})$ and blinded membership proofs $(\pi'_{t,x}, \pi'_{t',x})$ by creating a non-interactive discrete logarithm equality proof. We describe the four algorithms for our privately delegatable RSA accumulator scheme in Figure 4.

```

BlindMemProof(pp,  $\pi_{t,x}$ ):
1.  $\text{st} := r \xleftarrow{\$} \mathbb{Z}_{2\lambda}$ 
2.  $\pi'_{t,x} := \pi_{t,x} \cdot g^r$ 
3. return ( $\pi'_{t,x}, \text{st}$ )

BlindMemProofUpd(pp,  $S_t, \text{Acc}_t, \{x_i\}_{i=1}^k, \{y_i\}_{i=1}^l, \text{Acc}_{t'}, \pi'_{t,x}$ ):
1.  $S_{t'} = (S_t \cup \{x_i\}_{i=1}^k) \setminus \{y_i\}_{i=1}^l$ 
2.  $P_T \leftarrow \prod_{i=1}^k x_i, P_R \leftarrow \prod_{i=1}^l y_i$ 
3a.  $\Delta \leftarrow P_T \cdot (P_R^{-1} \bmod |\mathbb{G}|)$ 
3b.  $\Delta \leftarrow P_T (\in \mathbb{Z})$  // assuming no deletion
4.  $A = (\pi'_{t,x})^\Delta, B = g^\Delta$ 
5.  $\pi_1 \leftarrow \text{NIZK.Prove}(\mathcal{R}_{\text{DLEq}}, (\pi'_{t,x}, A, \text{Acc}_t, \text{Acc}_{t'}), \Delta)$ 
6.  $\pi_2 \leftarrow \text{NIZK.Prove}(\mathcal{R}_{\text{DLEq}}, (g, B, \pi'_{t,x}, A), \Delta)$ 
7.  $\pi'_{t',x} \leftarrow \{A, B\}, \text{aux} \leftarrow \{\pi_1, \pi_2\}$ 
8. return ( $\pi'_{t',x}, \text{aux}$ )

VerBlindMemProofUpd(pp,  $\text{Acc}_t, \text{Acc}_{t'}, \pi'_{t,x}, \pi'_{t',x}, \text{aux}$ ):
1.  $\{\pi_1, \pi_2\} \leftarrow \text{aux}, \{A, B\} \leftarrow \pi'_{t',x}$ 
2.  $b_1 := \text{NIZK.Verify}(\mathcal{R}_{\text{DLEq}}, (\pi'_{t,x}, A, \text{Acc}_t, \text{Acc}_{t'}), \pi_1)$ 
3.  $b_2 := \text{NIZK.Verify}(\mathcal{R}_{\text{DLEq}}, (g, B, \pi'_{t,x}, A), \pi_2)$ 
4. return  $b_1 \wedge b_2$ 

UnMemBlindProof( $\text{Acc}'_t, \text{st}, \pi'_{t',x}$ ):
1.  $\{A, B\} \leftarrow \pi'_{t',x}, r \leftarrow \text{st}$ 
2.  $\pi_{t',x} := A/B^r$ 
3. return  $\pi_{t',x}$ 

```

Fig. 4: The private delegation algorithms for *membership proof updates* in the RSA accumulator. Step 3a of `BlindProofUpd(·)` (typeset in blue) can be efficiently performed if the PSN knows the trapdoor of the group of unknown order, *i.e.*, its factorization in case of an RSA instantiation. This is reasonable when the PSN is also the accumulator manager.

Theorem 1 (RSA accumulator is privately delegatable for membership proofs). *Our construction in Figure 4 yields a secure, privately delegatable membership proof update scheme for the RSA with element additions and deletions under the strong RSA assumption and knowledge soundness of Figure 13.*

The proof can be found in Appendix B.1.

Non-membership proofs We describe the four efficient algorithms that allow a client to privately delegate the updates of their RSA accumulator non-membership proofs, *cf.* Figure 5. The intuition is that due to the hardness of

```

BlindNonMemProof(pp, x):
  1. if  $x \in S_t$  : return  $\perp$ 
  2.  $st := q \xleftarrow{\$} \text{Primes}(1^\lambda)$ 
  3.  $\overline{\pi_{t,x}'} := x \cdot q$ 
  4. return  $(\overline{\pi_{t,x}'}, st)$ 

BlindNonMemProofUpd(pp, Acct, {xi}i=1k, {yi}i=1l, Acct',  $\overline{\pi_{t,x}'}$ ):
  1.  $s^* := \prod_{s \in S_{t'}} s \bmod |\mathbb{G}|$ 
  2.  $\{a, b \bmod |\mathbb{G}|\} \leftarrow \text{Bezout}(s^*, \overline{\pi_{t,x}'})$ 
  3.  $\overline{\pi_{t',x}'} \leftarrow \{a, B := g^b\}, \text{aux} \leftarrow \emptyset$ 
  4. return  $(\overline{\pi_{t',x}'}, \text{aux})$ 

VerBlindNonMemProofUpd(pp, Acct',  $\overline{\pi_{t,x}'}, \overline{\pi_{t',x}'}, \text{aux}$ ):
  1.  $\{a, B\} \leftarrow \overline{\pi_{t',x}'}$ 
  2.  $y \leftarrow \overline{\pi_{t,x}'}$ 
  3. return 1 if  $g = \text{Acc}_{t'}^a B^y$ 

UnBlindNonMemProof(Acct', st,  $\overline{\pi_{t',x}'}$ ):
  1.  $\{a, B\} \leftarrow \overline{\pi_{t',x}'}$ 
  2.  $q \leftarrow st$ 
  3.  $B' := B^q$ 
  4. return  $\overline{\pi_{t',x}'} \leftarrow \{a, B'\}$ 

```

Fig. 5: The private delegation algorithms for updating *non-membership proofs* in the RSA accumulator. The steps 1 and 2 of **BlindNonMemProofUpd**(·) (typeset in blue) can be largely sped up if the PSN knows the trapdoor of the group of unknown order, *i.e.*, its factorization in case of an RSA instantiation. This is reasonable when the PSN is also the accumulator manager.

factoring, for a non-element x , *i.e.*, $x \notin S$, $x \cdot q$ hides x for appropriately large uniform random primes. Although in many RSA accumulator applications, elements are hashed to smaller primes, *i.e.*, of size 256-bits, thus, the proposed protocol requires the use of an H_{Primes} with a larger image space for privacy reasons. This is because factoring a 512-bit semiprime is considered highly practical [67].

Remark 2. The implicit assumption in the protocol presented in Figure 5 is that the size of the set that clients want to update their non-membership proofs for (denote this set as S' s.t. $S' \cap S = \emptyset$), has an exponential size in λ . In the stateless blockchain application of Section 1.2, elements are coin identifiers hashed to 256-bit primes, giving an effectively $\mathcal{O}(\lambda)$ -size element space; the weak notion suffices. In contrast, for certificate revocation lists with a bounded registry, this construction should not be used without additional precautions. Note that if $|S'| \in \text{poly}(\lambda)$, then it is trivial to break weak blinded non-membership proof indistinguishability: the server computes $\forall x' \in S' : \gcd(x', \overline{\pi_{t,x}'})$. If $\gcd(x', \overline{\pi_{t,x}'}) \neq$

1, the server found the element x' for which the clients obtained up-to-date non-membership proofs. Thus, this non-membership proof update delegation cannot satisfy blinded non-membership proof indistinguishability (*cf.* Definition 5). However, as we show next, it satisfies weak blinded non-membership proof indistinguishability, *cf.* Definition 6.

Theorem 2 (The RSA accumulator is privately delegatable for non-membership proofs). *Our construction in Figure 5 yields a privately delegatable non-membership proof update scheme for the RSA accumulator with element additions and deletions under the strong RSA and factoring assumptions in the random oracle model (ROM).*

The proof can be found in Appendix B.2.

5.4 Privately Delegating Proofs in Bilinear Accumulators

Next, we make the bilinear accumulator privately delegatable. Note that the following bilinear delegation protocols do not support element deletions, *i.e.*, we only support element additions. We leave this gap to future work.

Membership proofs The high-level idea of our protocol for privately delegating membership proofs in bilinear accumulators is as follows. Recall, in a bilinear accumulator $\forall s_j \in S \Leftrightarrow (X - s_j) \mid f(X)$, *i.e.*, $f(X) := \prod_{i=1}^n (X - s_i)$. Thus, the membership proof for an element s_j is the KZG commitment to the quotient polynomial $q(X) := \frac{f(X)}{X - s_j}$. Our idea is that the client sends a randomized common reference string crs' to the PSN, where each monomial in crs' is additionally raised to $rq(\tau)$, *i.e.*, $\text{crs}' := \{\text{KZG.Com}(rq(X)X^i)\}_{i=0}^k$, where τ is the trapdoor of the KZG CRS and r is a randomizer field element sampled by the client. The PSN uses crs' in the update algorithm to commit to the updated membership proof for the element s_j , *i.e.*, $q'(X) = \frac{f'(X)}{(X - s_j)} = q(X) \prod_{i=1}^k (X - s_i)$. The crs' allows the PSN to update the membership proof supporting up to k newly added elements. Finally, the client derandomizes the updated membership proof to obtain the latest valid membership proof for s_j . The private delegation protocol is detailed in Figure 6.

Preprocessing. At first glance, $\text{BlindMemProof}(\cdot)$ algorithm imposes an $\mathcal{O}(k)$ *online* computational cost on the client (*cf.* Figure 6) to generate the blinded reference string crs'_{s_j} , where k is the batch size of newly accumulated elements. However, we observe that the preparation of crs'_{s_j} is independent of the update elements $\{x_i\}_{i=1}^k$. Consequently, the client can compute this blinded CRS during an *offline, preprocessing* phase. In the online phase —when the accumulator transitions from t to t' —the heavy $\mathcal{O}(k)$ polynomial commitment is entirely delegated to the PSN. The client’s online computational cost is thereby reduced to verifying a constant-size PoEEq NIZK ($\mathcal{R}_{\text{PoEEq}}$) and performing a single unblinding exponentiation. This offline-online decoupling achieves a strictly $\mathcal{O}(1)$

```

BlindMemProof(pp, Acct, St, sj, πt,sj):
  1. st := r  $\xleftarrow{\$}$   $\mathbb{F}_p$ 
  2. q(X)  $\leftarrow \prod_{i=1, i \neq j}^n (X - s_i)$ 
  3. π't,sj  $\leftarrow \{\text{KZG.Com}(\text{crs}, rX^i q(X))\}_{i=0}^k$ 
  4. return (π't,sj, st)

BlindMemProofUpd(pp, Acct, {xi}i=1k, Acct', π't,sj):
  1. q*(X) :=  $\prod_{i=1}^k (X - x_i)$ , W := (π't,sj, π't',sj, Acct, Acct')
  2. crs'sj  $\leftarrow \{\pi'_{t,s_j}\}$ 
  3. π't',sj  $\leftarrow \text{KZG.Com}(\text{crs}'_{s_j}, q^*(X))$ 
  4. aux  $\leftarrow \text{NIZK.Prove}(\mathcal{R}_{\text{PoEEq}}, \mathbf{W}, \text{crs}'_{s_j}, q^*(X))$ 
  5. return (π't',sj, aux)

VerBlindMemProofUpd(π't,sj, π't',sj, Acct, Acct', aux):
  1. π  $\leftarrow$  aux
  2. b = NIZK.Verify( $\mathcal{R}_{\text{PoEEq}}$ , (π't,sj, π't',sj, Acct, Acct'), π)
  3. return b

UnBlindMemProof(Acct', st, π't',sj):
  1. r  $\leftarrow$  st
  2. return πt',sj  $\leftarrow (\pi'_{t',s_j})^{r^{-1} \bmod p}$ 

```

Fig. 6: The delegation algorithms for privately updating *membership proofs* in the bilinear accumulator.

online computational complexity for the client, making the delegation protocol highly practical for resource-constrained devices.

Theorem 3 (Bilinear accumulator is privately delegatable). *The construction in Figure 6 is a secure, privately delegatable membership proof update scheme for the bilinear accumulator, under q -Strong Diffie-Hellman (q -SDH) assumption in the algebraic group model and ROM.*

The proof can be found in Appendix B.3.

Non-membership proofs Unlike the membership case, the following protocol is interactive and requires k rounds between the client and the PSN – one per added element – making it suitable only for small offline update sizes. Recall that in the bilinear accumulator if $s_j \notin S_t \implies 0 \neq (s_t(X) \bmod (X - s_j))$. Thus, $s_t(X) = q_{t,s_j}(X)(X - s_j) + y_{t,s_j}$, where $y_{t,s_j} \neq 0$. That is, for s_j at time t we have: $\overline{\pi}_{t,s_j} := (\text{KZG.Com}(q_{t,s_j}(X), y_{t,s_j}))$ [30].

```

BlindNonMemProof(pp, Acct, sj,  $\overline{\pi_{t,s_j}}$ ):
1.  $\{g^{q_{t,s_j}(\tau)}, y_{t,s_j}\} \leftarrow \overline{\pi_{t,s_j}}$ 
2.  $(r_1, r_2) \xleftarrow{\$} \mathbb{F}_p, \text{st} \leftarrow (r_1, r_2, y_{t,s_j})$ 
3.  $\text{crs}' := \{g^{r_1 q_{t,s_j}(\tau)}, g^{r_1 \tau q_{t,s_j}(\tau)}\}$ 
4.  $\overline{\pi_{t,s_j}'} \leftarrow \{\text{crs}', r_2 \cdot y_{t,s_j}\}$ 
5. return  $(\overline{\pi_{t,s_j}'}, \text{st})$ 

BlindNonMemProofUpd(pp, Acct, sn+1, Acct+1,  $\overline{\pi_{t,s_j}'}$ ):
1.  $\{\text{crs}', y'_{t,s_j}\} \leftarrow \overline{\pi_{t,s_j}'}$ 
2.  $Q' := \text{KZG.Com}(\text{crs}', X - s_{n+1})$ 
3.  $y'_{t+1,s_j} := s_{n+1} \cdot y'_{t,s_j}, \text{aux} \leftarrow \{g_2^{s_{n+1}}\}$ 
4.  $\overline{\pi_{t+1,s_j}'} \leftarrow \{Q', y'_{t+1,s_j}\}$ 
5. return  $(\overline{\pi_{t+1,s_j}'}, \text{aux})$ 

VerBlindNonMemProofUpd( $\overline{\pi_{t,s_j}'}, \overline{\pi_{t+1,s_j}'}, s_{n+1}, \text{Acc}_{t+1}, \text{aux}$ ):
1.  $\{Q'_t, y'_{t,s_j}\} \leftarrow \overline{\pi_{t,s_j}'}, \{Q'_{t+1}, y'_{t+1,s_j}\} \leftarrow \overline{\pi_{t+1,s_j}'}, A \leftarrow \text{aux}$ 
2.  $b_1 := (y'_{t+1,s_j} \stackrel{?}{=} s_{n+1} \cdot y'_{t,s_j} \bmod p)$ 
3.  $\text{rhs} \leftarrow e(\text{Acc}_{t+1} \cdot Q'_{t+1} \cdot g_1^{\tau y'_{t,s_j}} / g_1^{y'_{t+1,s_j}}, g_2)$ 
4.  $b_2 := (e(\text{Acc}_t \cdot Q'_t \cdot g_1^{y'_{t,s_j}}, g_2^{\tau} / A) \stackrel{?}{=} \text{rhs})$ 
5. return  $b_1 \wedge b_2$ 

UnBlindNonMemProof(Acct, st,  $\overline{\pi_{t+1,s_j}'}$ ):
1.  $\{Q', y'_{t+1,s_j}\} \leftarrow \overline{\pi_{t+1,s_j}'}$ 
2.  $(r_1, r_2, y_{t,s_j}) \leftarrow \text{st}$ 
3.  $Q := (Q')^{r_1^{-1} \bmod p} \cdot g^{y_{t,s_j}}$ 
4.  $y_{t+1,s_j} := y_{t,s_j} s_j - y'_{t+1,s_j} / r_2$ 
5. return  $\overline{\pi_{t+1,s_j}} \leftarrow \{Q, y_{t+1,s_j}\}$ 

```

Fig. 7: The private delegation algorithms for updating *non-membership proofs* in the bilinear accumulator after a single element s_{n+1} is added to the accumulated set; $S_{t+1} = S_t \cup s_{n+1}$. If multiple elements are added during the offline period, the PSN and the client repeat this protocol sequentially for each added element.

One may be tempted to take a similar approach as in Figure 5, *i.e.*, using the PSN as a polynomial “Bézout-oracle”: the PSN could evaluate $\text{Bezout}(s_t(X), \cdot)$ for cleverly crafted input polynomials. This approach is not fruitful, as no matter how one blinds the desired query polynomial $(X - s_j)$ it seems difficult to obtain a meaningful protocol. For example, masking with a random r , *i.e.*, $r(X - s_j)$ seems to be at odds not only with privacy but also correctness, as $\text{Bezout}(s_t(X), \cdot)$ is non-linear. Masking $(X - s_j)$ with higher-degree polynomials is pointless as one can efficiently factor polynomials over finite fields due to Berlekamp [10]. Instead of the previous approach, we study how $(q_{t,s_j}(X), y_{t,s_j})$ must be updated after a *single element* s_{n+1} is added to the accumulator. That is,

$$\begin{aligned} s_{t+1}(X) &= s_t(X)(X - s_{n+1}) = \left(q_{t,s_j}(X)(X - s_j) + y_{t,s_j} \right)(X - s_{n+1}) = \\ &= q_{t+1,s_j}(X)(X - s_j) + y_{t+1,s_j} . \end{aligned}$$

The polynomial division computation (*i.e.*, $y_{t,s_j}(X - s_{n+1}) / (X - s_j) = y_{t,s_j} \wedge (y_{t,s_j}(X - s_{n+1}) \bmod (X - s_j)) \equiv (y_{t,s_j}s_j - y_{t,s_j}s_{n+1}))$) yields the following recurrence relation for updating non-membership proofs in bilinear accumulators after adding a single element to the accumulated set.

$$q_{t+1,s_j}(X) = q_{t,s_j}(X)(X - s_{n+1}) + y_{t,s_j} , \quad (5.3)$$

$$y_{t+1,s_j} = y_{t,s_j}s_j - y_{t,s_j}s_{n+1} . \quad (5.4)$$

The recurrence relations above suggest the following private delegation protocol. The PSN server and the client can engage in an interactive protocol during the update process for each element s_{n+1} added to the set S_t . In each round of the non-membership update protocol, the client sends a very short randomized crs' allowing the PSN to KZG-commit to a randomized version of $q_{t+1,s_j}(X)$. Similarly, the server can compute and return a randomized version of y_{t+1,s_j} . At the end of the protocol, the client can verify the correctness of the final received non-membership proof elements. The full protocol is detailed in Figure 7.

Theorem 4 (Bilinear accumulator is privately delegatable for non-membership proofs). *Our construction in Figure 7 yields a privately delegatable non-membership proof update scheme for the bilinear accumulator with additions under the q -SDH assumption in the ROM.*

The proof can be found in Appendix B.4.

6 Batching and Other Optimizations

6.1 Client-side Batching of Private Delegation

The accumulators we study in this work support constant-sized batch (non-)membership proofs [14,66]. These succinct batch proofs attest either that $B \subseteq S_t$, for membership, or that $B \cap S_t = \emptyset$, for non-membership. Thanks to the algebraic properties of these batch proofs, it is straightforward to modify our proof

update delegation protocols to the batch setting, *cf.* Figure 8. Essentially, the same blinding technique works that we developed in Sections 5.3 and 5.4. More precisely, we only need to change the semantics of the `Blind(Non)MemProof(·)` algorithms, *i.e.*, these algorithms work correctly even if we input batch (non-)membership proofs instead of single (non-)membership proofs. In this setting, the client either owns a batch of numerous set elements $B := \{s_i\}_{i=1}^m \subseteq S_t$ or non-set elements $B := \{s_i\}_{i=1}^m \cap S_t = \emptyset$ and wants to privately update a batch (non-)membership proof $\pi_{t,B}$ ($\overline{\pi}_{t,B}$) corresponding to B . We formalize this intuition in Figure 8.

RSA batch (non-)membership proof updates An RSA batch membership proof update for a batch B can be computed almost in the same fashion as in Figure 4. However, one needs to slightly modify the `Blind(Non)MemProof(·)` algorithm semantically, as it must now blind a batch membership proof $\pi_{t,B} := g^{\prod_{s_j \in S \setminus B} s_j}$, instead of a single membership proof, *cf.* Figure 8. A similar idea works, *cf.* Figure 8, for updating a batch RSA non-membership proof. Note, in this case, the batch size is leaked to the PSN, which, in certain applications, might be untenable. To avoid this, one may revisit our initial ideas in Section 1.4 to take another direction in designing an RSA non-membership proof delegation protocol.

Bilinear batch membership proof updates Observe that it is possible to adapt our membership proof delegation to the batch setting for the bilinear accumulator. In particular, we only need to modify the `BlindMemProof(·)` function, Figure 8. Crucially, the PSN cannot know whether it is updating a membership proof for a single element s_j or for a larger batch B .

6.2 Server-side Batching of Private Delegation

Imagine that there are numerous querying clients (denoted Q) who request proof updates for the same time period (*i.e.*, from t to t') for blinded membership proofs $\forall s_i \in Q : \pi'_{t,s_i}$. At a high-level, in both accumulators (*cf.* Figures 4 and 6), the server must compute $\forall s_i \in Q : (\pi'_{t,s_i})^\Delta$ for each queried element s_i , for some public exponent Δ , derived from the updated elements. The server's goal is to compute all updated proofs efficiently *e.g.*, in sublinear time in $|Q|$. However, as we show next, asymptotic server-side batching is impossible in the generic group model [64]. Thus, PSN operators cannot offer economies of scale in the naïve sense, which directly affects the business model of running a PSN, *cf.* Section 7.4.

Lemma 1 (Linear lower bound for component-wise exponentiation in the generic group model). *Let $\mathbb{G}$ be a cyclic group of prime order p in the generic group model. Let $g_1, \dots, g_n \in \mathbb{G}$ be independent random group elements and let $x \in \mathbb{Z}_p$ be known. Any generic algorithm that outputs $(g_1^x, g_2^x, \dots, g_n^x)$ must perform at least n oracle queries. In particular, the generic complexity of this computational task is $\Omega(n)$.*

We present the proof of this lemma in Appendix B.5.

```

BlindBatchRSAMemProof(pp,  $\pi_{t, \{s_j\}_{j=1}^m}$ ):
  1.  $st := r \xleftarrow{\$} \mathbb{Z}_{2\lambda}, B := \{s_j\}_{j=1}^m \subseteq S$ 
  2.  $\pi'_{t,B} := \pi_{t,B} \cdot g^r$ 
  3. return ( $\pi'_{t,B}, st$ )

BlindBatchRSANonMemProof(pp,  $\{s_j\}_{j=1}^m$ ):
  1. if  $\{s_j\}_{j=1}^m \cap S_t \neq \emptyset$  : return  $\perp$ 
  2.  $B := \prod_{j=1}^m s_j$ 
  3.  $st := q \xleftarrow{\$} \text{Primes}(1^\lambda)$ 
  4.  $\overline{\pi'_{t,B}} := B \cdot q$ 
  5. return ( $\overline{\pi'_{t,B}}, st$ )

BlindBatchBilinearMemProof(pp,  $\text{Acc}_t, S_t, \{s_j\}_{j=1}^m, \pi_{t, \{s_j\}_{j=1}^m}$ ):
  1.  $st := r \xleftarrow{\$} \mathbb{F}_p, B := \{s_j\}_{j=1}^m \subseteq S$ 
  2.  $q(X) \leftarrow \prod_{i=1, s_i \notin B}^n (X - s_i)$ 
  3.  $\text{crs}'_B \leftarrow \{\text{KZG.Com}(\text{crs}, rX^i q(X))\}_{i=0}^k$ 
  4.  $\pi'_{t,B} \leftarrow \{\text{crs}'_B\}$ 
  5. return ( $\pi'_{t,B}, st$ )

```

Fig. 8: The batch delegation algorithms for privately updating *batch (non-)membership proofs* in the RSA and bilinear accumulators. The rest of the algorithms remain the same as in the non-batch setting, *i.e.*, *cf.* Figures 4 to 6.

6.3 Batched Verification Proof

After this negative result, we now speed up the verification cost of m aux proofs issued to attest to the correctness of m proof delegation tasks. In particular, consider a client who blinds m membership proofs independently, but the PSN reuses the same update exponent Δ and produces *one NIZK proof* that all blinded membership proofs were updated consistently with the accumulator transition.

RSA accumulator For $\{x_j\}_{j=1}^m \subseteq S_t$ with membership proofs $\{\pi_{t,x_j}\}_{j=1}^m$, the client samples $\{r_j\}_{j=1}^m \xleftarrow{\$} \mathbb{Z}_{2\lambda}$ and sends blinded membership proofs $\pi'_{t,x_j} := \pi_{t,x_j} \cdot g^{r_j}$ for all $j \in [m]$ to the PSN. Given the update $(S_t, \text{Acc}_t) \rightarrow (S_{t'}, \text{Acc}_{t'})$ with exponent Δ (computed as in the base scheme), the PSN computes

$$A_j := (\pi'_{t,x_j})^\Delta \quad \text{for all } j \in [m], \quad B := g^\Delta.$$

To efficiently prove correctness, the PSN and the client derive random Fiat–Shamir challenges $\alpha_j := H(\text{Acc}_t, \text{Acc}_{t'}, B, \{\pi'_{t,x_i}\}_{i=1}^m, \{A_i\}_{i=1}^m, j)$ and aggregate the instances into $X := \prod_{j=1}^m (\pi'_{t,x_j})^{\alpha_j}$ and $Y := \prod_{j=1}^m A_j^{\alpha_j}$. The PSN then

returns $\{A_j\}_{j=1}^m, B$ together with a *constant-size* DLEq non-interactive zero-knowledge proof of knowledge for the aggregated relation

$$\exists \Delta : Y = X^\Delta \wedge \text{Acc}_{t'} = \text{Acc}_t^\Delta \wedge B = g^\Delta.$$

The client computes X and Y , verifies the DLEq NIZK proof, and then unblinds each updated blinded witness locally via $\pi_{t', x_j} := A_j / B^{r_j}$, recovering the updated RSA-accumulator membership proof $\pi_{t', x_j} = \pi_{t, x_j}^\Delta$ for every $j \in [m]$. By aggregating the relations, the aux proof size is reduced from $\mathcal{O}(m)$ to $\mathcal{O}(1)$ group/field elements. The soundness for this aggregated verification is based on the *low-order assumption* in the underlying group of unknown order, ensuring that adversarial error terms cannot be canceled out by wrapping around a small-order subgroup [14].

Bilinear accumulator In the bilinear setting, naively verifying m independent PoEEq proofs requires the client to compute m separate pairing equations. Given that the standard PoEEq relation (*cf.* Figure 14) uses an instance-dependent Fiat-Shamir challenge α_j that appears inside the $\mathbb{G}_2$ argument of the pairing (*i.e.*, $g_2^{\tau+\alpha_j}$), these pairings cannot be directly batched. To overcome this, we modify the PoEEq protocol to use a *shared* challenge for the entire batch. First, the PSN and the client derive a single global challenge $\alpha := H(\{a_j, f_j\}_{j=1}^m) \in \mathbb{F}_p^*$. For each instance $j \in [m]$, the PSN computes the quotient polynomial $h_j(X)$ such that $f_j(X) = (X + \alpha)h_j(X) + \beta_j$, and sends the proof elements $Q_j := g_1^{h_j(\tau)}$ to the client. Because α is shared, the $\mathbb{G}_2$ pairing argument $g_2^{\tau+\alpha}$ is now identical across all m verification equations. This thus enables the client to verify the entire batch using a random linear combination. In particular, the client derives random Fiat-Shamir weights $\gamma_j := H'(\alpha, \{Q_i\}_{i=1}^m, j) \in \mathbb{F}_p^*$ and computes the aggregated elements in $\mathbb{G}_1$:

$$Q_{\text{agg}} := \prod_{j=1}^m Q_j^{\gamma_j} \quad \text{and} \quad A_{\text{agg}} := \prod_{j=1}^m (a_j \cdot g_1^{-\beta_j})^{\gamma_j}.$$

The client then accepts the batch if and only if the following single pairing equation holds: $e(Q_{\text{agg}}, g_2^{\tau+\alpha}) \stackrel{?}{=} e(A_{\text{agg}}, g_2)$.

7 Implementation and Performance Evaluation

7.1 Theoretical Performance

Observe that all client algorithms in the RSA (non-)membership delegation protocols, *i.e.*, $\text{BlindProof}(\cdot)$, $\text{VerBlind(Non)MemProofUpd}(\cdot)$, and $\text{UnBlind(Non)MemProof}(\cdot)$ run in $\mathcal{O}(1)$ time (*cf.* Table 3), which is independent of the updated set size as desired in real-world applications. The linear $\mathcal{O}(k)$ computational cost of $\text{Blind(Non)MemProof}(\cdot)$ in the bilinear membership proof delegation protocol could be offloaded to a preprocessing phase. Unsurprisingly, *cf.* Lemma 1, the asymptotic cost of the PSN server is $\mathcal{O}(k)$ in all our private delegation protocols.

Private Delegation Protocol	$ \pi'_{t,x} $	$ \text{aux} $	BlindProof	BlindProofUpd	VerBlindProofUpd	UnBlindProof
Merkle tree mem. [18]	$\mathcal{O}(\log_2^2(kd)) \mathbb{G} $	$\emptyset$	N/A	$\approx k(d \mathbb{Z}_p + d \mathbb{G} + H)$	$\mathcal{O}(\log_2(kd))\mathbb{G}$	N/A
RSA mem. (Figure 4)	$ \mathbb{G}_U $	$4 \mathbb{G}_U + 2 \mathbb{Z}_\lambda $	$\mathbb{G}_U$	$2k\mathbb{Z}_\lambda + 6\mathbb{G}_U + 2H_{\text{Primes}}$	$8\mathbb{G}_U + 2H_{\text{Primes}}$	$\mathbb{G}_U$
RSA non-mem. (Figure 5)	$ \mathbb{Z}_\lambda $	$\emptyset$	$\mathbb{Z}_\lambda$	$k\mathbb{Z}_\lambda + \mathbb{G}_U$	$2\mathbb{G}_U$	$\mathbb{G}_U$
Bilinear mem. (Figure 6)	$(k+1) \mathbb{G} $	$4 \mathbb{G} + \mathbb{Z}_p $	$(k+1)\mathbb{G}$	$(4k+2)\mathbb{G} + (2k+1)\mathbb{Z}_p + H$	$3P$	$\mathbb{G} + \mathbb{Z}_p$
Bilinear non-mem. (Figure 7)	$2k \mathbb{G} + k \mathbb{Z}_p $	$k \mathbb{G}_2 $	$2k\mathbb{G} + k\mathbb{Z}_p$	$2k\mathbb{G} + k\mathbb{Z}_p + k\mathbb{G}_2$	$2kP + 3k\mathbb{G} + k\mathbb{Z}_p$	$2k\mathbb{G} + 3k\mathbb{Z}_p$

Table 3: Theoretical performance of our private (non-)membership proof update delegation algorithms for the RSA and bilinear accumulators. $\mathbb{G}, \mathbb{G}_U, P, H, \mathbb{Z}_\lambda$ denote elliptic curve, unknown order group operations (*i.e.*, RSA, class groups), pairing, hashing and finite field operations, respectively. Recall, k denotes the #elements added to S_t in the offline phase and d , the depth of the Merkle tree, *i.e.*, $d = \mathcal{O}(\log_2(|S_t|))$. For this comparison, we instantiated [18] with the Nova folding scheme and Spartan decider SNARK [51]. Note that [18] provides no public implementation, precluding direct empirical comparison.

7.2 Our Implementation

In the publicly available implementation ⁷ of our protocols, all code is implemented in Rust, using the Arkworks library [3] and the Zengo class group library [28]. The criterion-rs crate was used for all benchmarks. We chose the security parameter as $\lambda = 128$. Following NIST guidelines, this amounts to using moduli N of bit-length of 3072 [59]. Following the work of Dobson *et al.* [32], we chose the discriminant Δ for class groups of imaginary quadratic fields to be 6784 bits long, providing λ bit security for class groups [33]. Accordingly, we chose the BLS12-381 curve that is currently known to provide roughly 128-bit security [17]. Interestingly, the Rust num_bigint library always chooses the most efficient integer multiplication algorithm (*i.e.*, textbook, Karatsuba [47], or Toom-Cook multiplication algorithms [50]) depending on the input integer sizes. Similarly, division is implemented using Knuth’s algorithm; when dividing by linear factors, we employ synthetic division [35]. Our hash-to-prime function $H_{\text{Prime}}(\cdot)$ is implemented with the image space of the first $2^{2\lambda}$ prime numbers to provide a collision resistance of λ -bits (and λ -bit soundness in the applied NIZKs compiled with the Fiat-Shamir heuristic [36]). Applying the prime number theorem, we set the image space of $H_{\text{Prime}}(\cdot)$ to $\{0, 1\}^{2\lambda + \log(2\lambda)}$, *i.e.*, in our case $\{0, 1\}^{262}$. We instantiate $H_{\text{Prime}}(\cdot)$ using a deterministic hash-and-increment construction, starting from $H_{\text{SHA256}}(\cdot)$ we test the primality of odd candidates with the Miller-Rabin probabilistic primality testing algorithm [60] using 64 iterations. All experiments are run on a consumer laptop with 16 GB RAM, and an Apple M2 processor with 8 cores. Each experiment was run 50 times.

7.3 Performance Evaluation

Client Runtime Analysis In Table 4, we provide our benchmarks for the constant-time client algorithms. Observe in Table 4 that all client algorithms

⁷ See: <https://github.com/GlaszBoti/private-accumulator-proof-delegation>.

Algorithm	RSA		Class Group		Bilinear	
	Avg(ms)	Var(ms)	Avg(ms)	Var(ms)	Avg(ms)	Var(ms)
BlindMemProof($\cdot$)	18.82	4.60	56.68	15.37	0.94	0.00
UnBlindMemProof($\cdot$)	10.71	0.00	46.58	0.00	0.21	0.00
BlindNonMemProof($\cdot$)	13.15	0.60	8.03	0.69	0.75	0.00
UnBlindNonMemProof($\cdot$)	6.25	0.00	40.82	0.01	0.32	0.00
VerBlindMemProofUpd($\cdot$)	90.95	0.01	513.39	2.65	8.23	0.00
VerBlindNonMemProofUpd($\cdot$)	20.29	0.01	161.65	0.30	8.31	0.00

Table 4: Performance evaluation of the constant-time client algorithms in the private (non-)membership proof update protocols for the RSA (instantiated with $\mathbb{Z}_N^*$ and class groups of imaginary quadratic fields) and bilinear accumulators.

run in the millisecond range, showcasing the practicality of our protocols for clients, one of our initial design goals. Moreover, the results show that the bilinear accumulator-based constructions are significantly faster, with sub-millisecond running time for most operations. The RSA accumulator instantiated with class groups avoids trusted setups, but incurs substantial overhead for all our algorithms due to larger group sizes and the slower Abelian group law for composing binary quadratic forms. The low variance across all schemes and algorithms provides evidence of stable and deterministic behavior of our implementations.

Server Runtime Analysis In Figure 9a, we measure the total runtime of private (non-)membership proof updates as the set S_t grows. Our private delegation algorithms for the bilinear accumulator significantly outperform other private membership update algorithms, achieving at least $2\times$ faster runtime when adding up to 1024 elements. Similarly, for non-membership proof updates, the bilinear accumulator consistently outperforms every RSA accumulator instantiation. Interestingly, the trapdoor setting of RSA (*i.e.*, when the factorization is known to the server) achieves constant-time, *i.e.*, $\mathcal{O}(1)$ updates. Figure 15 investigates which cryptographic operations (*e.g.*, hashing, exponentiation (multi-scalar multiplication), NIZK proof generation, etc.) constitute a bottleneck for the server during private proof update delegation.

Baseline Comparison: Privacy Overhead We consider the accumulator scheme’s (non-)membership proof computational cost as baseline. In Figure 9b, we evaluate the computational overhead incurred by our private (non-)membership proof update algorithms as the size of the updates grows. Formally, we define the privacy overhead as the ratio

$$\kappa(k) = \frac{t(\text{Blind(Non)MemProofUpd}(\cdot), k)}{t(\text{(Non)MemProofCreate}(\cdot), k)}, \quad (7.1)$$

where $t(\cdot, k)$ denotes the running time of an algorithm when k elements were added to the accumulator in the offline phase. Intuitively, a client can request

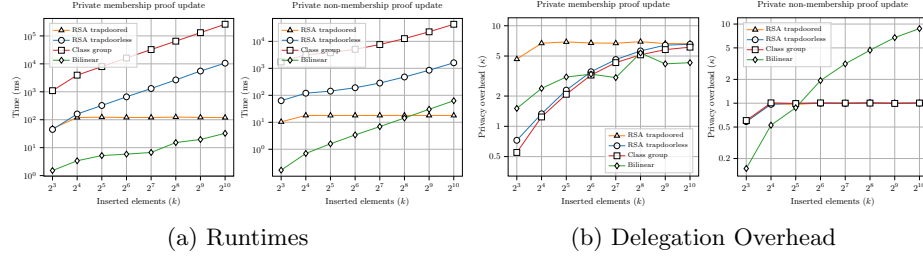


Fig. 9: PSN server running times in the (non-)membership proof update algorithms ($\text{Blind}(\text{Non})\text{MemProofUpd}(\cdot)$). Private update overhead is defined as the ratio of the running times of the $\text{Blind}(\text{Non})\text{MemProofUpd}(\cdot)$ and $(\text{Non})\text{MemCreate}(\cdot)$ algorithms.

a fresh (non-)membership proof from the server without privacy or use private proof updates that may incur a performance overhead for the PSN.

Observe that the overhead of the RSA accumulator’s private proof delegation algorithms $\kappa(\cdot)$ quickly converges. In case of membership proof updates, this is expected as exponentiation is the dominant cost, and BlindMemProofUpd performs six exponentiations with $k\lambda$ -bit exponent against the single exponentiation of $(n+k)\lambda$ -bit exponent, so $\frac{6k}{n+k} \rightarrow 6$ as $k \rightarrow \infty$ where n is the number of elements in the accumulator before the updates. Similarly, non-membership algorithms converge to one as both NonMemProofCreate and $\text{BlindNonMemProofUpd}$ do one extended GCD and exponentiation. The bilinear accumulator incurs a high privacy overhead attributed to the already efficient proof creation and interactive design of non-membership proof updates. Overall, the data demonstrate that the RSA-based instantiation provides the lowest overhead for privately delegated (non-)membership proof updates.

7.4 Case Study: PSN for Stateless Blockchains

We study the communication and operational costs of running a PSN server for the stateless blockchain use case [14,24,65] in Figure 10. At the time of writing, the hourly running cost of a t3.medium AWS instance with 8 threads is $\text{cost} = 0.052$ USD/hour [2]. In Bitcoin, the coin age distribution (the time elapsed between the creation and spending of a UTXO) follows a power-law distribution, *i.e.*, $p(x) = Cx^{-\alpha}$, where $C = e^{-1.4159} = 0.243$, $\alpha = 0.9550$ as measured in [63]. Thus, the average coin age in Bitcoin is $\text{avg}(p(x)) \approx 162$ days. For simplicity, for a given stateless blockchain throughput y (in transactions per second (tps)), we treat that between each coin spending there are $y \cdot \text{avg}(p(x))$ new elements inserted into the accumulator representing the set of unspent coins in the cryptocurrency system. In case of the bilinear accumulator, we assume the system uses the longest available crs today, which has a length of 2^{28} [69]. Thus, the PSN can support the outsourcing of a membership proof update iff $y \cdot \text{avg}(p(x)) \leq 2^{28}$. In this case, the operational cost is

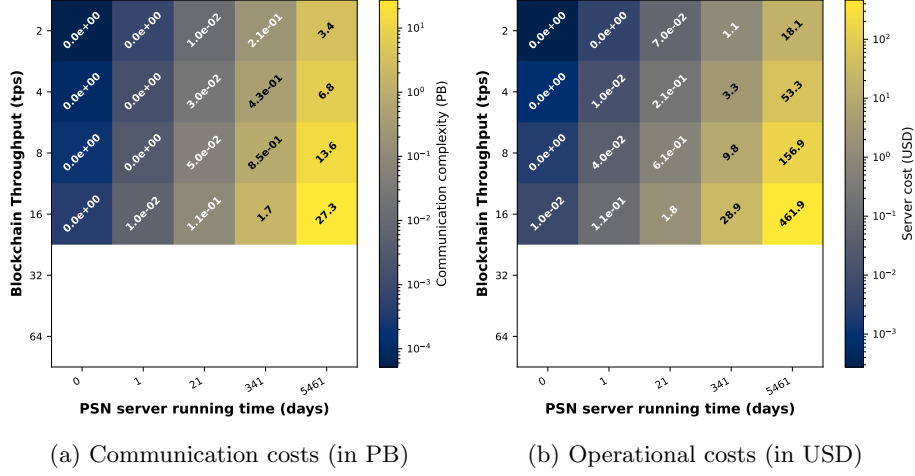


Fig. 10: Stateless Blockchain case study. Communication and operational costs of a PSN for a blockchain throughput and uptime with the bilinear accumulator.

$cost \cdot z \cdot (y \cdot \text{avg}(p(x))) \cdot t(\text{BlindMemProofUpd}(\cdot), y \cdot \text{avg}(p(x)))$ assuming that the PSN serves its clients for z days and the running times $t(\cdot, \cdot)$ were approximated from the measurements of Figure 9 using linear regression ($R^2 = 0.95$). Figure 10 suggests that operating a PSN with real-world workloads has low economic costs in the stateless blockchain application.

8 Open Problems and Future Directions

In this work, we presented efficient algorithms that allow an intermittently connected, resource-constrained client to privately delegate its (non-)membership proof updates to an untrusted server. We built and evaluated efficient schemes for the RSA and bilinear accumulators. We leave several open problems and foresee promising research directions for future work.

Limitations We acknowledge the shortcomings of our private (non-)membership proof delegation protocols, *e.g.*, RSA non-membership proof updates achieve only weak privacy (*cf.* Section 5.3), and the bilinear non-membership proofs only admit interactive private delegation protocols with rounds proportional to the number of updates (*cf.* Section 5.4). These design constraints, however, inherently stem from the algebraic structure of the underlying RSA and bilinear accumulator schemes we build upon. We aim for our work to be an initiative for building efficient private delegation protocols for accumulator schemes and possibly other authenticated data structures. We hope that the (in)ability to build efficient private delegation algorithms better informs researchers and practitioners to keep private delegation in mind when deploying accumulator schemes.

in large-scale, real-world applications. Moreover, we highlight the following as potential directions for future research.

Delegation-friendly accumulator schemes Our work highlights that we still lack the “perfect” cryptographic accumulator scheme that could be readily applied and deployed in real-world applications serving millions of resource-constrained devices. Designing an efficient, universal, dynamic, batchable, delegation-friendly accumulator scheme is still a major open problem.

Post-quantum privately delegatable accumulators None of our delegation schemes provides post-quantum security. Thus, the imminent post-quantum transition motivates the need to design private delegation schemes for post-quantum secure accumulators [49,55]. We leave the construction of post-quantum variants of our schemes as an exciting research question.

Other authenticated data structures Our primary focus in this work was to design private proof delegation for universal, dynamic accumulators. One could explore other authenticated data structures such as vector commitments (VC), polynomial commitment schemes, or authenticated dictionaries. For instance, we believe similar techniques could work for the Hyperproofs VC scheme [65]. The authors in [46] construct a key-value commitment (KVC) scheme in which a dictionary (a set of key-value pairs $D = \{(k_i, v_i)\}$) is represented directly as a sum of partial fractions. It seems in reach to design a private delegation scheme for the (non-)membership proofs in their KVC scheme with $\mathcal{O}(1)$ server complexity.

Multi-server proof delegation In real-world deployments, it may be the case that multiple servers store different disjoint subsets of the update. Thus, it is promising to extend our protocols to a multi-server setting, *e.g.*, for better liveness. A basic design could be to have PSNs share a zero secret in the preprocessing phase, and collaboratively generate the proof update (blinded by a zero share). We leave the details of this protocol to future work.

Acknowledgements. We are grateful to Ivan Homoliak, Alin Tomescu, and Steve Thakur for insightful conversations. István András Seres was supported by the Ministry of Culture and Innovation and the National Research, Development, and Innovation Office within the Quantum Information National Laboratory of Hungary (Grant No. 2022-2.1.1-NL-2022-00004).

References

1. Hamza Abusalah, Gaspard Anthoine, Gennaro Avitabile, and Emanuele Giunta. Lower bounding update frequency in short accumulators and vector commitments. Cryptology ePrint Archive, 2025.
2. Amazon Web Services. Aws pricing calculator: Amazon ec2 estimate. <https://calculator.aws/#/createCalculator/ec2-enhancement>, 2026. Accessed Apr. 2026.
3. arkworks contributors. arkworks zkspark ecosystem. <https://arkworks.rs>, 2022.

4. Foteini Baldimtsi, Jan Camenisch, Maria Dubovitskaya, Anna Lysyanskaya, Leonid Reyzin, Kai Samelin, and Sophia Yakubov. Accumulators with applications to anonymity-preserving revocation. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 301–315, San Jose, CA, USA, 2017. IEEE.
5. Foteini Baldimtsi, Ioanna Karantaidou, and Srinivasan Raghuraman. Oblivious accumulators. In Qiang Tang and Vanessa Teague, editors, *Public-Key Cryptography – PKC 2024*, pages 99–131, Cham, 2024. Springer Nature Switzerland.
6. Anaïs Barthoulot, Olivier Blazy, and Sébastien Canard. Cryptographic accumulators: New definitions, enhanced security, and delegatable proofs. In *Cryptology in Africa Conference*, Lecture Notes in Computer Science, pages 94–119, Cham, Switzerland, 2024. Springer.
7. Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy (S&P 2014)*, pages 459–474, San Jose, CA, USA, 2014. IEEE.
8. Josh Benaloh and Michael de Mare. One-way accumulators: A decentralized alternative to digital signatures. In *Advances in Cryptology – EUROCRYPT 1993*, volume 765 of *Lecture Notes in Computer Science*, pages 274–285, Berlin, Heidelberg, 1994. Springer.
9. Josh Benaloh and Michael De Mare. One-way accumulators: A decentralized alternative to digital signatures. In *Advances in Cryptology – EUROCRYPT 1993*, volume 765 of *Lecture Notes in Computer Science*, pages 274–285, Berlin, Heidelberg, 1994. Springer.
10. Elwyn R. Berlekamp. Factoring polynomials over finite fields. *Bell System Technical Journal*, 46(8):1853–1859, 1967.
11. Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, pages 329–349. ACM, New York, NY, USA, 2019.
12. Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. Verifiable delay functions. In *Annual international cryptology conference*, pages 757–788. Springer, 2018.
13. Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 41–55, Berlin, Heidelberg, 2004. Springer.
14. Dan Boneh, Benedikt Bünz, and Ben Fisch. Batching techniques for accumulators with applications to iops and stateless blockchains. In *Advances in Cryptology – CRYPTO 2019*, Lecture Notes in Computer Science, pages 561–586, Cham, Switzerland, 2019. Springer.
15. Dan Boneh and Matt Franklin. Identity-based encryption from the weil pairing. In *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229, Berlin, Heidelberg, 2001. Springer.
16. Joseph Bonneau, Jessica Chen, Miranda Christ, and Ioanna Karantaidou. Merkle mountain ranges are optimal: On witness update frequency for cryptographic accumulators. In *Advances in Cryptology – CRYPTO 2025*, volume 14025 of *Lecture Notes in Computer Science*, pages 170–202, Cham, Switzerland, 2025. Springer.
17. Sean Bowe. Bls12-381: New zk-snark elliptic curve construction. *Zcash Company blog*, URL: <https://z.cash/blog/new-snark-curve>, 22, 2017.
18. Sean Bowe and Ian Miers. A note on notes: Towards scalable anonymous payments via evolving nullifiers and oblivious synchronization. *Cryptology ePrint Archive*, 2025.

19. Jan Camenisch, Markulf Kohlweiss, and Claudio Soriente. An accumulator based on bilinear maps and efficient revocation for anonymous credentials. In *International Workshop on Public Key Cryptography (PKC 2009)*, volume 5443 of *Lecture Notes in Computer Science*, pages 481–500, Berlin, Heidelberg, 2009. Springer.
20. Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Advances in Cryptology – EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*, pages 93–118, Berlin, Heidelberg, 2001. Springer.
21. Jan Camenisch and Anna Lysyanskaya. Dynamic accumulators and application to efficient revocation of anonymous credentials. In *Advances in Cryptology – CRYPTO 2002*, volume 2442 of *Lecture Notes in Computer Science*, pages 61–76, Berlin, Heidelberg, 2002. Springer.
22. Jan Camenisch and Anna Lysyanskaya. A signature scheme with efficient protocols. In *Security in Communication Networks (SCN 2002)*, volume 2576 of *Lecture Notes in Computer Science*, pages 268–289, Berlin, Heidelberg, 2003. Springer.
23. David Chaum and Torben Pryds Pedersen. Wallet databases with observers. In *Advances in Cryptology – CRYPTO 1992*, volume 740 of *Lecture Notes in Computer Science*, pages 89–105, Berlin, Heidelberg, 1992. Springer.
24. Alexander Chepur, Charalampos Papamanthou, Shrawan Srinivasan, and Yupeng Zhang. Edrax: A cryptocurrency with stateless transaction validation. *Cryptology ePrint Archive*, 2018.
25. Céline Chevalier, Fabien Laguillaumie, and Damien Vergnaud. Privately outsourcing exponentiation to a single server: cryptanalysis and optimal constructions. *Algorithmica*, 83(1):72–115, 2021.
26. Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *Journal of the ACM (JACM)*, 45(6):965–981, 1998.
27. Miranda Christ and Joseph Bonneau. Limits on revocable proof systems, with implications for stateless blockchains. In *International Conference on Financial Cryptography and Data Security (FC 2023)*, volume 13924 of *Lecture Notes in Computer Science*, pages 54–71, Cham, Switzerland, 2023. Springer.
28. class contributors. Rust library for building iqc. <https://github.com/ZenGo-X/class>, 2019.
29. Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sub-linear online time. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT 2020)*, volume 12105 of *Lecture Notes in Computer Science*, pages 44–75, Cham, Switzerland, 2020. Springer.
30. Ivan Damgård and Nikos Triandopoulos. Supporting non-membership proofs with bilinear-map accumulators. *Cryptology ePrint Archive*, 2008.
31. Sergi Delgado-Segura, Cristina Pérez-Solà, Guillermo Navarro-Arribas, and Jordi Herrera-Joancomartí. Analysis of the bitcoin utxo set. In *Financial Cryptography and Data Security*, volume 10957 of *Lecture Notes in Computer Science*, pages 78–91, Cham, Switzerland, 2018. Springer.
32. Samuel Dobson, Steven Galbraith, and Benjamin Smith. Trustless unknown-order groups. *arXiv*, 2022.
33. Samuel Dobson, Steven D. Galbraith, and Benjamin Smith. Trustless unknown-order groups. *Cryptology ePrint Archive*, Paper 2020/196, 2020.
34. European Parliament and the Council of the European Union. Regulation (eu) 2024/1183 of the european parliament and of the council of 11 april 2024 amending regulation (eu) no 910/2014 as regards establishing the european digital identity framework. *Official Journal of the European Union*, L 2024/1183, 30 April 2024,

- April 2024. ELI: <http://data.europa.eu/eli/reg/2024/1183/oj>. Accessed: 2026-07-22.
35. Lianghuo Fan. A generalization of synthetic division and a general theorem of division of polynomials. *Mathematical Medley*, 30(1):30–37, June 2003.
 36. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – EUROCRYPT 1986*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Berlin, Heidelberg, 1987. Springer.
 37. Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, and Ahmadreza Rahimi. Registration-based encryption: Removing private-key generator from ibe. In *Theory of Cryptography Conference (TCC 2018)*, volume 11239 of *Lecture Notes in Computer Science*, pages 689–718, Cham, Switzerland, 2018. Springer.
 38. Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, Ahmadreza Rahimi, and Sruthi Sekar. Registration-based encryption from standard assumptions. In *IACR international workshop on public key cryptography*, pages 63–93. Springer, 2019.
 39. Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, STOC ’09, pages 169–178, New York, NY, USA, 2009. ACM.
 40. Noemi Glaeser, Dimitris Kolonelos, Giulio Malavolta, and Ahmadreza Rahimi. Efficient registration-based encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’23, pages 1065–1079, New York, NY, USA, 2023. ACM.
 41. Golem Network. Golem network. <https://golem.network/>, 2026. Accessed: 2026-04-16.
 42. Tiantian Gong, Aniket Kate, Alexandros Psomas, and Athina Terzoglou. V3rified: Revelation vs non-revelation mechanisms for decentralized verifiable computation. *arXiv*, 2024.
 43. Iftach Haitner, Jonathan J. Hoch, and Gil Segev. A linear lower bound on the communication complexity of single-server private information retrieval. In *Theory of Cryptography Conference (TCC 2008)*, volume 4948 of *Lecture Notes in Computer Science*, pages 445–464, Berlin, Heidelberg, 2008. Springer.
 44. Susan Hohenberger and Anna Lysyanskaya. How to securely outsource cryptographic computations. In *Theory of Cryptography Conference (TCC 2005)*, volume 3378 of *Lecture Notes in Computer Science*, pages 264–282, Berlin, Heidelberg, 2005. Springer.
 45. Samuel Jaques, Hart Montgomery, and Michael Lodder. Allosaur: Accumulator with low-latency oblivious sublinear anonymous credential updates with revocations. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pages 1708–1723, New York, NY, USA, 2024. ACM.
 46. Charanjit S. Jutla, Rohit Nema, and Arnab Roy. Partial fraction techniques for cryptography. Cryptology ePrint Archive, 2025.
 47. Anatolii A. Karatsuba and Yuri Ofman. Multiplication of many-digital numbers by automatic computers. *Doklady Akademii Nauk SSSR*, 145(2):293–294, 1962.
 48. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology – ASIACRYPT 2010*, *Lecture Notes in Computer Science*, pages 177–194, Berlin, Heidelberg, 2010. Springer.
 49. Victor Youdom Kemmoe, Anna Lysyanskaya, and Ngoc Khanh Nguyen. Lattice-based accumulator and application to anonymous credential revocation. In Shi

- Bai and Edoardo Persichetti, editors, *Public-Key Cryptography – PKC 2026*, pages 404–436, Cham, 2026. Springer Nature Switzerland.
50. Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, Reading, MA, USA, 3 edition, 1997.
 51. Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Advances in Cryptology – CRYPTO 2022*, pages 359–388, Cham, 2022. Springer Nature Switzerland.
 52. Jiangtao Li, Ninghui Li, and Rui Xue. Universal accumulators with efficient non-membership proofs. In *International Conference on Applied Cryptography and Network Security*, pages 253–269. Springer, 2007.
 53. Yiping Ma, Ke Zhong, Tal Rabin, and Sebastian Angel. Incremental Offline/Online pir. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 1741–1758, Boston, MA, USA, 2022. USENIX Association.
 54. Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. CONIKS: Bringing key transparency to end users. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 383–398, Washington, D.C., USA, 2015. USENIX Association.
 55. Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology – EUROCRYPT 1987*, Lecture Notes in Computer Science, pages 369–378, Berlin, Heidelberg, 1987. Springer.
 56. Ian Miers, Christina Garman, Matthew Green, and Aviel D. Rubin. Zerocoin: Anonymous distributed e-cash from bitcoin. In *2013 IEEE Symposium on Security and Privacy (S&P 2013)*, pages 397–411, Berkeley, CA, USA, 2013. IEEE.
 57. Lan Nguyen. Accumulators from bilinear pairings and applications. In *Cryptographers’ Track at the RSA Conference (CT-RSA 2005)*, volume 3376 of *Lecture Notes in Computer Science*, pages 275–292, Berlin, Heidelberg, 2005. Springer.
 58. Charalampos Papamanthou. *Cryptography for Efficiency: New Directions in Authenticated Data Structures*. PhD thesis, Brown University, 2011.
 59. W Timothy Polk, Donna F Dodson, William E Burr, Hildegard Ferraiolo, and David Cooper. Cryptographic algorithms and key sizes for personal identity verification. *NIST Special Publication*, 800(78-4):1–19, 2015.
 60. Michael O Rabin. Probabilistic algorithm for testing primality. *Journal of number theory*, 12(1):128–138, 1980.
 61. Jothi Rangasamy and Lakshmi Kuppusamy. Revisiting single-server algorithms for outsourcing modular exponentiation. In *International Conference on Cryptology in India (INDOCRYPT 2018)*, volume 11263 of *Lecture Notes in Computer Science*, pages 3–20, Cham, Switzerland, 2018. Springer.
 62. Leonid Reyzin and Sophia Yakoubov. Efficient asynchronous accumulators for distributed pki. In *Security and Cryptography for Networks*, Lecture Notes in Computer Science, pages 292–309, Cham, Switzerland, 2016. Springer.
 63. Mario Schröck. Time is money: How coin age shapes bitcoin’s spending patterns. <https://insights.glassnode.com/time-is-money-how-coin-age-shapes-bit-coins-spending-patterns/>, December 2024. Glassnode Insights blog.
 64. Victor Shoup. Lower bounds for discrete logarithms and related problems. In *Advances in Cryptology – EUROCRYPT 1997*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266, Berlin, Heidelberg, 1997. Springer.
 65. Shravan Srinivasan, Alexander Chepurnoy, Charalampos Papamanthou, Alin Tomescu, and Yupeng Zhang. Hyperproofs: Aggregating and maintaining proofs in vector commitments. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3001–3018, 2022.

66. Steve Thakur. Batching non-membership proofs with bilinear accumulators. *Cryptography ePrint Archive*, 2019.
67. Luke Valenta, Shaanan Cohney, Alex Liao, Joshua Fried, Satya Bodduluri, and Nadia Heninger. Factoring as a service. In *Financial Cryptography and Data Security*, volume 9603 of *Lecture Notes in Computer Science*, pages 321–338, Berlin, Heidelberg, 2016. Springer.
68. Giuseppe Vitto and Alex Biryukov. Dynamic universal accumulator with batch update over bilinear groups. In *Cryptographers’ Track at the RSA Conference (CT-RSA 2022)*, *Lecture Notes in Computer Science*, pages 395–426, Cham, Switzerland, 2022. Springer.
69. Faxing Wang, Shaanan Cohney, and Joseph Bonneau. Sok: Trusted setups for powers-of-tau strings. In *International Conference on Financial Cryptography and Data Security (FC 2025)*, volume 14513 of *Lecture Notes in Computer Science*, pages 197–218, Cham, Switzerland, 2025. Springer.
70. Yujue Wang, Qianhong Wu, Duncan S. Wong, Bo Qin, Sherman S. M. Chow, Zhen Liu, and Xiao Tan. Securely outsourcing exponentiations with single untrusted program for cloud storage. In *European Symposium on Research in Computer Security (ESORICS 2014)*, volume 8712 of *Lecture Notes in Computer Science*, pages 326–343, Cham, Switzerland, 2014. Springer.

A Additional Background

A.1 The RSA and bilinear accumulators

We recall the algorithms of the universal, dynamic RSA [9,52] and bilinear accumulators [57,30] in Figures 11 and 12.

A.2 The KZG Polynomial Commitment scheme

KZG Polynomial Commitments. The Kate–Zaverucha–Goldberg (KZG) commitment scheme [48] allows committing to a polynomial over a finite field while later enabling succinct proofs of its evaluations.

KZG.Setup($1^\lambda, n$) $\rightarrow$ **crs**. Let $\tau \xleftarrow{\$} \mathbb{F}_p$. Compute the structured reference string $\text{crs} = (g_1, g_1^\tau, g_1^{\tau^2}, \dots, g_1^{\tau^n}, g_2, g_2^\tau)$. Output **crs**. These public parameters allow users to commit to polynomials of degree at most n without knowing the trapdoor τ itself.

KZG.Com(**crs**, f) $\rightarrow C$. Let $f(X) = \sum_{i=0}^d a_i X^i$ be a polynomial with $d \leq n$. Compute $C = g_1^{f(\tau)} = g_1^{\sum_{i=0}^d a_i \tau^i}$. Output the commitment C . To commit to $f(X)$, the committer evaluates the polynomial at the hidden value τ in the exponent. Using the **crs** powers of τ , this exponentiation can be computed efficiently without knowing τ .

KZG.Open(**crs**, f, z) $\rightarrow \pi$. Compute the evaluation $y = f(z)$. Compute the quotient polynomial $q(X) = \frac{f(X) - y}{X - z}$. Compute the proof $\pi = g_1^{q(\tau)}$. Output (y, π) .

KZG.Vrfy(**crs**, C, z, y, π) $\rightarrow \{0, 1\}$ Accept if and only if $e(C/g_1^y, g_2) = e(\pi, g_2^\tau/g_2^z)$. Otherwise reject. The verifier checks a pairing equation encoding the identity $f(\tau) - y = (\tau - z)q(\tau)$ in the exponent.

Setup(λ): 1. $\mathbb{G} \xleftarrow{\$} \mathbf{GGen}(\lambda)$ 2. $g \xleftarrow{\$} \mathbb{G}$ 3. return $\mathbb{G}, g$ Add(Acc_t, S_t, x): 1. if $x \in S_t$: return Acc_t 2. else: 3. $S_{t+1} \leftarrow S_t \cup \{x\}$ 4. $u \leftarrow x$ 5. $\text{Acc}_{t+1} \leftarrow \text{Acc}_t^x$ 6. return Acc_t^x, u Del(Acc_t, S_t, x): 1. if $x \notin S$: return Acc_t 2. else: 3. $S_{t+1} \leftarrow S_t \setminus \{x\}$ 4a. $\text{Acc}_{t+1} \leftarrow g^{\prod_{s \in S_{t+1}} s}$ 4b. $\text{Acc}_{t+1} \leftarrow \text{Acc}_t^{x^{-1} \bmod \mathbb{G} }$ 5. $u \leftarrow \{x, \text{Acc}_t, \text{Acc}_{t+1}\}$ 6. return Acc_{t+1}, u	MemProofCreate(Acc_t, S_t, x): 1a. $\pi_{t,x} \leftarrow g^{\prod_{s \in S_t, s \neq x} s}$ 1b. $\pi_{t,x} \leftarrow \text{Acc}_t^{x^{-1} \bmod \mathbb{G} }$ 2. return $\pi_{t,x}$ NonMemProofCreate(Acc_t, S_t, x): 1. $s^* \leftarrow \prod_{s \in S_t} s \bmod \mathbb{G} $ 2. $a, b \bmod \mathbb{G} \leftarrow \mathbf{Bezout}(s^*, x)$ 3. $B \leftarrow g^b$ 4. return $\overline{\pi_{t,x}} \leftarrow \{a, B\}$ MemVer($\text{Acc}_t, \pi_{t,x}, x$): 1. return 1 if $(\pi_{t,x})^x = \text{Acc}_t$ NonMemVer($\text{Acc}_t, \overline{\pi_{t,x}}, x$): 1. $\{a, B\} \leftarrow \overline{\pi_{t,x}}$ 2. return 1 if $\text{Acc}_t^a B^x = g$
--	--

Fig. 11: The algorithms of the universal, dynamic RSA accumulator [9,52]. Certain algorithms can be significantly sped up whenever the group order is known to a trusted accumulator manager (written in blue).

Security properties. The KZG commitment scheme satisfies correctness, (polynomial) binding, and evaluation binding. Intuitively, polynomial binding ensures that a polynomial commitment cannot be opened to two different polynomials. Evaluation binding ensures that at any point $z \in \mathbb{F}_p$, the prover can only open the committed polynomial to a single evaluation point y . Finally, correctness ensures that an honest opening proof is always accepted by the verifier. For the formal security definitions, the reader is referred to [48].

A.3 NIZK for NP-statement in Groups of Unknown Order

Discrete logarithm equality (sometimes Chaum-Pedersen proof) (DLEq) [23,14].

$$\mathcal{R}_{\text{DLEq}} = \{((g, u, h, v) \in \mathbb{G}^4; w \in \mathbb{Z}) : u = g^w \wedge v = h^w\}. \quad (\text{A.1})$$

Setup (λ, n):	MemProofCreate ($\text{Acc}_t, S_t, s_t(X), s_j$):
1. $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p) \xleftarrow{\\$} \mathbf{GGen}(\lambda)$ 2. $g_1 \xleftarrow{\\$} \mathbb{G}_1, g_2 \xleftarrow{\\$} \mathbb{G}_2, s_0(X) \leftarrow 1$ 3. $\text{pp} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, s_0(X))$ 4. $\text{crs} := (\{g_1^{\tau^i}\}_{i=0}^n, g_2, g_2^\tau)$ for $\tau \xleftarrow{\\$} \mathbb{F}_p$ 5. return (pp, crs)	1. $q_{t,s_j}(X) \leftarrow s_t(X)/(X - s_j)$ 2. $\pi_{t,s_j} \leftarrow \text{KZG.Com}(q_{t,s_j}(X))$ 3. return π_{t,s_j}
Add ($\text{Acc}_t, S_t, s_t(X), s_j$):	NonMemProofCreate ($\text{Acc}_t, S_t, s_t(X), s_j$):
1. if $s_j \in S_t$: return Acc_t 2. else: 3. $S_{t+1} \leftarrow S_t \cup \{s_j\}$ 4. $s_{t+1}(X) \leftarrow s_t(X) \cdot (X - s_j)$ 5. $\text{Acc}_{t+1} \leftarrow \text{KZG.Com}(s_{t+1}(X))$ 6. return $\text{Acc}_{t+1}, s_{t+1}(X)$	1. $q_{t,s_j}(X) \leftarrow s_t(X)/(X - s_j)$ 2. $y_{t,s_j} \leftarrow s_t(X) \bmod (X - s_j)$ 3. $B_{t,s_j} \leftarrow \text{KZG.Com}(q_{t,s_j}(X))$ 4. return $\overline{\pi_{t,s_j}} \leftarrow \{y_{t,s_j}, B_{t,s_j}\}$
Del (Acc_t, S_t, s_j):	MemVer ($\text{Acc}_t, \pi_{t,s_j}, s_j$):
1. if $s_j \notin S_t$: return Acc_t 2. else: 3. $S_{t+1} \leftarrow S_t \setminus \{s_j\}$ 4. $s_{t+1}(X) \leftarrow s_t(X)/(X - s_j)$ 5. $\text{Acc}_{t+1} \leftarrow \text{KZG.Com}(s_{t+1}(X))$ 6. return $\text{Acc}_{t+1}, s_{t+1}(X)$	1. return 1 if $e(\pi_{t,s_j}, g_2^\tau / g_2^{s_j}) = e(\text{Acc}_t, g_2)$
	NonMemVer ($\text{Acc}_t, \overline{\pi_{t,s_j}}, s_j$):
	1. $\{y_{t,s_j}, B_{t,s_j}\} \leftarrow \overline{\pi_{t,s_j}}$ 2. $lhs \leftarrow e(B_{t,s_j}, g_2^{\tau-s_j})e(g_1, g_2)^{y_{t,s_j}}$ 3. return 1 if $lhs = e(\text{Acc}_t, g_2)$

Fig. 12: The algorithms of the bilinear accumulator [57].

B Deferred Security Proofs

B.1 RSA Membership Proof Delegation

Theorem 5 (RSA accumulator is privately delegatable for membership proofs). *Our construction in Figure 4 yields a secure, privately delegatable membership proof update scheme for the RSA accumulator under the strong RSA assumption and knowledge soundness of Figure 13.*

Proof. Correctness. For any accumulated set S_t and any element $x \in S_t$, an honestly run proof update should yield a valid membership proof for the updated accumulator $\text{Acc}_{t'}$ that is accepted with overwhelming probability.

Let the accumulator at time t be $\text{Acc}_t = g^{\prod_{s \in S_t} s}$. For $x \in S_t$ the corresponding membership proof is $\pi_{t,x} = g^{\prod_{s \in S_t \setminus \{x\}} s}$. Then, an honestly generated proof update on the blinded witness $\pi'_{t,x} = \pi_{t,x} \cdot g^r$ yields

$$(\pi'_{t',x}, \text{aux}) = ((\pi'_{t,x})^{\prod_{i=1}^k x_i}, ((\pi'_{t,x})^r, g^r, z)) \quad , \quad (\text{B.1})$$

NIZK.Prove (pp, $\mathcal{R}_{\text{DLEq}}$, $(\hat{g}, u, h, v), w$):	NIZK.Verify (pp, $\mathcal{R}_{\text{DLEq}}$, $(\hat{g}, u, h, v), \pi$):
1. $g^* \leftarrow g^w$	1. $(Q_1, Q_2, Q_3, g^*, r) \leftarrow \pi$
2. $e \leftarrow \text{HPrimes}(\hat{g}, h, u, v, g^*)$	2. $e \leftarrow \text{HPrimes}(\hat{g}, h, u, v, g^*)$
3. $(q, r) \leftarrow \mathbb{Z}^2$ s.t. $w = q \cdot e + r$ for $r \in [e]$	3. return 1 if $r \in [e] \wedge u = \hat{g}^r \cdot Q_1^e \wedge$ $\wedge v = h^r \cdot Q_2^e \wedge Q_3^e \cdot g^r = g^*$
4. $Q_1 \leftarrow \hat{g}^q, Q_2 \leftarrow h^q, Q_3 \leftarrow g^q$	
5. return $\pi := (Q_1, Q_2, Q_3, g^*, r)$	

Fig. 13: A non-interactive zero-knowledge (NIZK) proof system for the relation $\mathcal{R}_{\text{DLEq}}$ using the Fiat-Shamir heuristic instantiated in a group of unknown order [14,66]. Note that the proof system has constant-size proofs (four $\mathbb{G}$ elements and a λ -bit integer) and a constant-time (six λ -bit exponentiations in $\mathbb{G}$) verifier.

where the current set of accumulated values is $S_{t'} = (S_t \cup \{x_i\}_{i=1}^k)$. The unblind algorithm outputs

$$\pi_{t',x} = \frac{\pi'_{t',x}}{(g^\Delta)^r} = \pi_{t,x}^\Delta. \quad (\text{B.2})$$

Finally, we verify the unblinded, updated membership proof

$$(\pi_{t',x})^x = \left(g^{\prod_{s \in S_t/\{x\}} s}\right)^{x\Delta} = g^{(\prod_{s \in S_t} s) \cdot \Delta} = \text{Acc}_{t'}, \quad (\text{B.3})$$

which always leads to acceptance, thus completing the proof of correctness.

Blinded Membership Proof Indistinguishability. Consider the blinded membership proof indistinguishability game in Figure 3. The adversary $\mathcal{A}$ sends x_0, x_1 , after which the challenger $\mathcal{C}$ samples a random bit $b \xleftarrow{\$} \{0, 1\}$ and responds with the blinded membership proof $\pi'_{1,x_b} = \pi_{1,x_b} \cdot g^r$, where r is freshly sampled for each query. If r is sampled uniformly at random, then $\pi_{1,x_b} \cdot g^r$ is uniform in $\mathbb{G}$, and thus, independent of the challenger's bit b . In other words, we have information-theoretic indistinguishability. Therefore,

$$\text{Adv}_{\mathcal{A}}^{\text{ind}} = \left| \Pr[b' = b] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

Delegation Soundness. Towards contradiction, let us assume that there exists an adversary $\mathcal{A}$ that can output a blinded membership proof $(\pi'_{t',x}, \text{aux})$ for an element $x \in S_t$ with non-negligible probability s.t.,

$$\begin{aligned} & \text{MemVer}(\text{UnBlindProof}(\text{Acc}_t, \text{st}, \pi'_{t',x})) = \\ & = 0 \wedge \text{VerBlindProofUpd}(\text{pp}, \text{Acc}_{t'}, \pi'_{t,x}, \pi'_{t',x}, \text{aux}) = 1. \end{aligned}$$

Since the applied NIZK for the discrete logarithm equality NP-relation is an argument of knowledge, the extractor $\mathcal{E}$ outputs the triplet (a, b, z) satisfying the NIZK.Verify function with non-negligible probability. Thus, there exists a

NIZK.Prove(pp, $\mathcal{R}_{\text{PoEEq}}$, (g, u, h, v) , crs, $\text{crs}_g, q^*(X)$):

1. $\tilde{g} \leftarrow \text{KZG.Com}(\text{crs}, q^*(X))$
2. $\alpha \leftarrow H(g, u, h, v)$
3. $h(X) \leftarrow q^*(X)/(X + \alpha)$
4. $\beta \leftarrow q^*(X) \bmod (X + \alpha)$
5. $Q_g \leftarrow \text{KZG.Com}(\text{crs}_g, h(X))$
6. $Q_h \leftarrow \text{KZG.Com}(\text{crs}, S_t(X) \cdot h(X))$
7. $\hat{g} \leftarrow \text{KZG.Com}(\text{crs}, h(X))$
8. **return** $\pi := (\tilde{g}, Q_g, Q_h, \hat{g}, \beta)$

NIZK.Verify(pp, $\mathcal{R}_{\text{PoEEq}}$, (g, u, h, v) , crs, crs_g, π):

1. $(\tilde{g}, Q_g, Q_h, \hat{g}, \beta) \leftarrow \pi$
2. $\alpha \leftarrow H(\hat{g}, g, u, h, v)$
3. $eq_1 := \mathbf{e}(Q_g, g_2^{\tau+\alpha}) \cdot \mathbf{e}(g^\beta, g_2) \stackrel{?}{=} \mathbf{e}(u, g_2)$
4. $eq_2 := \mathbf{e}(Q_h, g_2^{\tau+\alpha}) \cdot \mathbf{e}(h^\beta, g_2) \stackrel{?}{=} \mathbf{e}(v, g_2)$
5. $eq_3 := \mathbf{e}(\hat{g}, g_2^{\tau+\alpha}) \cdot \mathbf{e}(g_1^\beta, g_2) \stackrel{?}{=} \mathbf{e}(\tilde{g}, g_2)$
6. **return** 1 if $eq_1 \wedge eq_2 \wedge eq_3$

Fig. 14: A non-interactive zero-knowledge (NIZK) proof system for the relation $\mathcal{R}_{\text{PoEEq}}$ using the Fiat–Shamir heuristic.

value Δ s.t. $(g, \text{Acc}_t, \pi'_{t,x}) = (g^\Delta, \text{Acc}_t^\Delta, (\pi'_{t,x})^\Delta)$. Unblinding the blinded RSA membership proof update yields

$$\pi'_{t',x} = \frac{\pi'_{t,x}}{B^r}.$$

Substituting this into the MemVer algorithm, we have $\pi_{t',x}^x = \text{Acc}_{t'}$ as desired. Thus, the only way an adversary could produce a valid BlindProofUpd that leads to an invalid unblinded proof is by breaking the knowledge soundness of the applied NIZK. $\square$

B.2 RSA Non-membership Proof Delegation

Theorem 6 (The RSA accumulator is privately delegatable for non-membership proofs). *Our construction in Figure 5 yields a secure, privately delegatable non-membership proof update scheme for the RSA accumulator under strong RSA and factoring assumptions in the ROM.*

Proof. Correctness. For $x \notin S_t$, we have $\overline{\pi'_{t',x}} = \{a, g^b\}$, for $(a, b) \leftarrow \text{Bezout}(s^*, x \cdot q)$. The unblinded non-membership proof is obtained by $B' := (g^b)^q$, while the Bézout-coefficient a remains unchanged. By construction, $as^* + b(xq) = 1$. Due to the commutativity of integer multiplication, we also have $as^* + bq(x) = 1$. Thus, (a, bq) are the Bézout-coefficients of s^* and x , as required.

Weak Blinded Non-Membership Proof Indistinguishability. Let us assume that there is an efficient adversary $\mathcal{A}$ that on input $N := \overline{\pi_{t,x}'}'$ outputs with non-negligible probability $\frac{1}{\text{poly}(\lambda)}$ to which element $x \in \{0, 1\}^\lambda$ the blinded non-membership proof belongs, *i.e.*, $N = x \cdot q$ for some primes $x, q \in \text{Primes}(\lambda)$. Adversary $\mathcal{A}$ can be used by another machine $\mathcal{B}$ to break the factoring assumption. Machine $\mathcal{B}$ on any input N should just forward its input to $\mathcal{A}$ who factors it, by assumption, with non-negligible probability. Adversary $\mathcal{B}$ clearly has the same time-complexity and success probability as $\mathcal{A}$ in its own weak blinded non-membership proof indistinguishability game.

Delegation soundness. Towards contradiction, assume that there exists an adversary $\mathcal{A}$ that outputs $x \notin S_t$ and a blinded non-membership proof $\overline{\pi_{t',x}'}' = \{a, B\}$ for the updated accumulator $\text{Acc}_{t'}$ such that

$$g = \text{Acc}_{t'}^a \cdot B^y$$

holds in the blinded verification (for some exponent y used in the blinded update protocol), but the RSA accumulator's non-membership proof verification fails for the corresponding unblinded proof, *i.e.*,

$$g \neq \text{Acc}_{t'}^a \cdot (B^q)^x .$$

However, by construction of the blinded update, we have $y = q \cdot x$. Substituting this into the first equation yields

$$g = \text{Acc}_{t'}^a \cdot B^{qx} = \text{Acc}_{t'}^a \cdot (B^q)^x ,$$

which directly contradicts the inequality above. Hence, both equations cannot hold simultaneously with non-negligible probability, and no such adversary $\mathcal{A}$ can exist. $\square$

B.3 Bilinear Membership Proof Delegation

Theorem 7 (Bilinear accumulator is privately delegatable). *The construction in Figure 6 is a secure, privately delegatable membership proof update scheme for the bilinear accumulator under q -SDH in the algebraic group model and ROM.*

Proof. Correctness. By the definition of the blinding algorithm, we have that $\text{crs}'_j = \{\text{KZG.Com}(rX^j q(X))\}_{i=1}^k$. Let us denote the polynomial of the newly added elements $\{x_i\}_{i=1}^k$ in the monomial bases as $q^*(X) = \prod_{i=1}^k (X - x_i) = \sum_{j=0}^k a_j X^j$. Using the homomorphic properties of KZG, in the $\text{BlindProofUpd}(\cdot)$ algorithm, we have

$$\begin{aligned} \pi'_{t',s_j} &= \prod_{j=0}^k (\text{crs}'_j)^{a_j} = \prod_{j=0}^k \text{KZG.Com}(rX^j q(X))^{a_j} = \\ &= \prod_{j=0}^k \text{KZG.Com}(a_j \cdot rX^j q(X)) = \text{KZG.Com}\left(\sum_{j=0}^k a_j \cdot X^j r q(X)\right) , \end{aligned}$$

factoring out the update independent polynomial $rq(X)$ yields the following.

$$\begin{aligned} \text{KZG.Com} \left(\sum_{j=0}^k a_j \cdot X^j rq(X) \right) &= \text{KZG.Com} \left(rq(X) \cdot \sum_{j=0}^k a_j X^j \right) = \\ &= \text{KZG.Com} (rq(X)q^*(X)) . \end{aligned}$$

By the unblinding algorithm, $\text{UnBlindProof}(\cdot)$, we have that

$$(\pi'_{t',s_j})^{-r} = \text{KZG.Com} (rq(X)q^*(X))^{-r} = \text{KZG.Com} (q(X)q^*(X)) = \pi_{t',s_j} ,$$

as desired. Therefore, π_{t',s_j} is indeed the KZG commitment to the new quotient polynomial $q(X)q^*(X)$.

Blinded Membership Proof Indistinguishability. In the blinded membership proof indistinguishability game, the adversary chooses s_0, s_1 and sees a crs randomized by the challenger with $r \in_R \mathbb{F}_p$, *i.e.*, $\text{crs}' := \{\text{KZG.Com}(rX^i q_b(X))\}_{i=1}^k$, where $q_b(X) = \frac{f(X)}{X-s_b}$, for $b \in_R \{0, 1\}$. Since $q_b(\tau) \neq 0$, let $s := r \cdot q_b(\tau)$. Since in both cases s is uniform, *i.e.*, $s \sim U(\mathbb{F}_p)$, and multiplication by a non-zero constant in $\mathbb{F}_p$ is a permutation, in both cases, the adversary sees the distribution $\{g^{s\tau^i}\}_{i=1}^k$ for a secret $s \in \mathbb{F}_p$. Thus, regardless of which element has been chosen by the challenger, $\mathcal{A}$ sees perfectly indistinguishable distributions $\{g^{s\tau^i}\}_{i=1}^k \approx \{g^{s^*\tau^i}\}_{i=1}^k$ for the challenger chosen $(s, s^*) \in_R \mathbb{F}_p^2$.

Delegation Soundness. The NIZK proof system applied for the NP-language $\mathcal{R}_{\text{PoEEq}}$ was shown to be knowledge sound in the algebraic group model [66]. Thus, if aux is verified for the statement of the polynomial exponentiation equality $\mathcal{R}_{\text{PoEEq}}$, then for the public instance $(\pi'_{t,s_j}, \pi'_{t',s_j}, \text{Acc}_t, \text{Acc}_{t'})$, there is a polynomial $q^*(X) \in \mathbb{F}_p^{\leq k}[X]$ s.t.

$$(\pi'_{t,s_j})^{q^*(\tau)} = \pi'_{t',s_j} \wedge (\text{Acc}_t)^{q^*(\tau)} = \text{Acc}_{t'} , \quad (\text{B.4})$$

where τ is the trapdoor of the KZG crs . We see in Equation (B.4) that $q^*(X)$ encodes the update polynomial $\prod_{i=1}^k (X - x_i)$ between the two accumulator digests Acc_t and $\text{Acc}_{t'}$. Thus,

$$(\pi'_{t',s_j})^{r^{-1} \bmod p} = \left(g^{rq(\tau)q^*(\tau)} \right)^{r^{-1} \bmod p} = \pi_{t',s_j} \quad (\text{B.5})$$

is a valid membership proof as required. $\square$

B.4 Bilinear Non-membership Proof Delegation

Theorem 8 (Bilinear accumulator is privately delegatable for non-membership proofs). *Our construction in Figure 7 yields a secure, privately delegatable non-membership proof update scheme for the bilinear accumulator under q -SDH in the ROM.*

Proof. Correctness. First, the client generates a short randomized common reference string $\text{crs}' := \{g^{r_1 q_{t,s_j}(\tau)}, g^{r_1 \tau q_{t,s_j}(\tau)}\}$ which is used by the server to KZG-commit to the polynomial $X - s_{n+1}$, i.e., $Q' := \text{KZG.Com}(\text{crs}', X - s_{n+1})$. Then, in the $\text{UnBlindNonMemProof}(\cdot)$ algorithm, the client rerandomizes Q' and obtains $Q = (Q')^{r_1^{-1} \bmod p} \cdot g^{y_{t,s_j}} = g^{\tau q_{t,s_j}(\tau) - s_{n+1} q_{t,s_j}(\tau) + y_{t,s_j}} = \text{KZG.Com}(\text{crs}, (X - s_{n+1})q_{t,s_j}(X) + y_{t,s_j})$, which is exactly the desired quotient polynomial for the updated non-membership proof, cf. Equation (5.3). Similarly, the client blinds y_{t,s_j} to $y'_{t,s_j} = r_2 \cdot y_{t,s_j}$. In turn, the server computes $y'_{t+1,s_j} = s_{n+1} \cdot y'_{t,s_j}$ which is derandomized at the end of the protocol by the client in the $\text{UnBlindNonMemProof}(\cdot)$ algorithm as $y_{t,s_j} s_j - y'_{t+1,s_j} / r_2 = y_{t,s_j} s_j - s_{n+1} \cdot y'_{t,s_j} / r_2 = y_{t,s_j} s_j - s_{n+1} \cdot r_2 \cdot y_{t,s_j} / r_2 = y_{t,s_j} s_j - s_{n+1} y_{t,s_j}$, which is the desired scalar element required by the updated non-membership proof recurrence relation, cf. Equation (5.3).

Blinded Non-Membership Proof Indistinguishability. Given two blinded membership proofs $\overline{\pi'_{t,s_0}} = \{\text{crs}'_0, r_{0,2} \cdot y_{t,s_0}\}$ and $\overline{\pi'_{t,s_1}} = \{\text{crs}'_1, r_{1,2} \cdot y_{t,s_1}\}$ for $\text{crs}'_0 := \{g^{r_{0,1} q_{t,s_0}(\tau)}, g^{r_{0,1} \tau q_{t,s_0}(\tau)}\}$ and $\text{crs}'_1 := \{g^{r_{1,1} q_{t,s_1}(\tau)}, g^{r_{1,1} \tau q_{t,s_1}(\tau)}\}$, it is clear that $\overline{\pi'_{t,s_0}}$ is information-theoretically indistinguishable from $\overline{\pi'_{t,s_1}}$, since, first, $r_{0,2} \cdot y_{t,s_0} \approx_s r_{1,2} \cdot y_{t,s_1}$ for uniformly random $(r_{0,2}, r_{1,2}) \leftarrow \mathbb{F}_p^2$, and second, $\text{crs}'_0 \approx_s \text{crs}'_1$, because $r_{0,1}$ and $r_{1,1}$ act as random permutations on crs_0 and crs_1 .

Delegation Soundness. Consider any PPT adversary (PSN) that, given $\overline{\pi'_{t,s_j}}$ and the public update $(s_{n+1}, \text{Acc}_t, \text{Acc}_{t+1})$, outputs $(\overline{\pi'_{t+1,s_j}}, \text{aux})$ such that $\text{VerBlindNonMemProofUpd}(\overline{\pi'_{t,s_j}}, \overline{\pi'_{t+1,s_j}}, s_{n+1}, \text{Acc}_{t+1}, \text{aux}) = 1$. We show that, except with negligible probability, the unblinded witness $\overline{\pi_{t+1,s_j}}$ returned by $\text{UnBlindNonMemProof}$ is a valid non-membership proof for $s_j \notin S_{t+1}$ under Acc_{t+1} . The pairing check can be rewritten, using bilinearity and $\text{Acc}_{t+1} = \text{Acc}_t^{\tau - s_{n+1}}$, as an equality of exponents in $\mathbb{G}_1$ linking Q'_t and Q'_{t+1} via the same factor $(\tau - s_{n+1})$. By soundness of the accumulator and binding of KZG commitments, any deviation of Q'_{t+1} from the honest updated value (i.e., the commitment to $r_1 q_{t+1,s_j}(\tau)$, where $q_{t+1,s_j}(X) = (X - s_{n+1})q_{t,s_j}(X) + y_{t,s_j}$) would violate this pairing equation except with negligible probability. Finally, in $\text{UnBlindNonMemProof}$ the client computes

$$Q := (Q'_{t+1})^{r_1^{-1}} \cdot g^{y_{t,s_j}} = g^{q_{t+1,s_j}(\tau)},$$

so $\overline{\pi_{t+1,s_j}} = (Q, y_{t+1,s_j})$ is exactly the standard non-membership witness for $s_j \notin S_{t+1}$ under Acc_{t+1} . Therefore, the probability that a PPT adversary produces an accepting transcript for $\text{VerBlindNonMemProofUpd}$ while the resulting unblinded witness is invalid is negligible. $\square$

B.5 Server-side Batching

The following theorem states that asymptotic server-side batching for our *membership proof delegation* protocols (cf. Figures 4 and 6), i.e., $\mathcal{O}(n)$: sublinear in the number of delegation requests, is impossible in the generic group model. Specifically, we assume there are n users trying to update their membership proofs for the same transition $t \rightarrow t'$. Then, the membership proof update, at a

high level, translates to exponentiating n random group $\{g_i\}_{i=1}^n$ with the same exponent x . As we show next, this computation needs $\Omega(n)$ exponentiation.

Lemma 2 (Linear lower bound for component-wise exponentiation in the generic group model). *Let $\mathbb{G}$ be a cyclic group of prime order p in the generic group model. Let $g_1, \dots, g_n \in \mathbb{G}$ be independent random group elements and let $x \in \mathbb{Z}_p$ be known. Any generic algorithm that outputs*

$$(g_1^x, g_2^x, \dots, g_n^x)$$

must perform at least n oracle queries. In particular, the generic complexity of this task is $\Omega(n)$.

Proof. Fix a generator g of $\mathbb{G}$ and write

$$g_i = g^{\alpha_i},$$

where $(\alpha_1, \dots, \alpha_n) \xleftarrow{\$} \mathbb{Z}_p^n$ are independent, uniform random exponents unknown to the algorithm. By the standard analysis of generic algorithms [64], every group element computed by a generic algorithm can be written as

$$g^{L(\alpha_1, \dots, \alpha_n)}$$

where L is a linear polynomial

$$L(\alpha_1, \dots, \alpha_n) = \sum_{j=1}^n a_j \alpha_j$$

with coefficients $a_j \in \mathbb{Z}_p$ known to the algorithm.

Thus, each known group element corresponds to a vector

$$a = (a_1, \dots, a_n) \in \mathbb{Z}_p^n.$$

Let $V_t \subseteq \mathbb{Z}_p^n$ denote the linear span of all coefficient vectors corresponding to elements known by the algorithm after t oracle queries.

Initially the algorithm knows $g_1, \dots, g_n$, corresponding to the vectors

$$e_1, \dots, e_n,$$

where e_i is the i -th standard basis vector. Hence

$$V_0 = \text{span}\{e_1, \dots, e_n\}.$$

Each oracle query (group multiplication or inversion) produces at most one new group element, whose coefficient vector is a linear combination of previously known vectors. Consequently, after t oracle queries the algorithm knows at most $n + t$ group elements.

The desired outputs are

$$g_i^x = g^{x\alpha_i},$$

which correspond to the vectors

$$xe_1, \dots, xe_n.$$

Since the α_i are independent random variables, these elements are independent group elements. Therefore, the algorithm must explicitly produce the encoding of each g_i^x .

As each oracle query can produce at most one new group element, producing n distinct outputs requires at least n oracle queries. Thus, any generic algorithm computing $(g_1^x, \dots, g_n^x)$ requires at least n oracle queries, proving the $\Omega(n)$ lower bound. $\square$

It would be fascinating to prove lower bounds for the server complexity of updating n client *non-membership proofs* in the standard or even just in the generic or algebraic group models. We leave this seemingly harder problem to future work.

C Extended Performance Evaluation

Next, we detail some performance measurements that we did not include in the main text due to space constraints.

C.1 Runtime Breakdown of (Non-)Membership Proof Updates

Figure 15 illustrates what percentage of the runtime is spent on various cryptographic operations, *i.e.*, hashing, exponentiation (multi-scalar multiplication (MSM) in case of the bilinear accumulator), and NIZK proving during private (non-)membership proof updates when updating for a single element (*i.e.*, $k = 1$). We note that in the case of private membership proof updates, the size of the exponent grows linearly in k (*cf.* Section 7.1), making exponentiation the dominant operation. For all instantiations of the RSA accumulator, we expect that, as $k \rightarrow \infty$, the dominant cost will be the exponentiation costs. Luckily, specialized hardware chips that can compute exponentiation in groups of unknown order (*i.e.*, in RSA and class groups) extremely fast are already available due to verifiable delay function deployments [12]. These hardware chips could be leveraged by PSNs for these computational tasks.

C.2 Case Study: PSN for Stateless Blockchains from the RSA Accumulator

We study the operation and communication costs of deploying a PSN for the stateless blockchain use case, whenever the unspent coins are accumulated using the *trapdoorless* RSA accumulator, *cf.* Figure 16. Observe in Figure 16 that the RSA membership proof delegation protocol that we developed can support arbitrarily scalable stateless blockchains, since there is no public parameter that would limit the desired proof update functionality. However, we also see that the

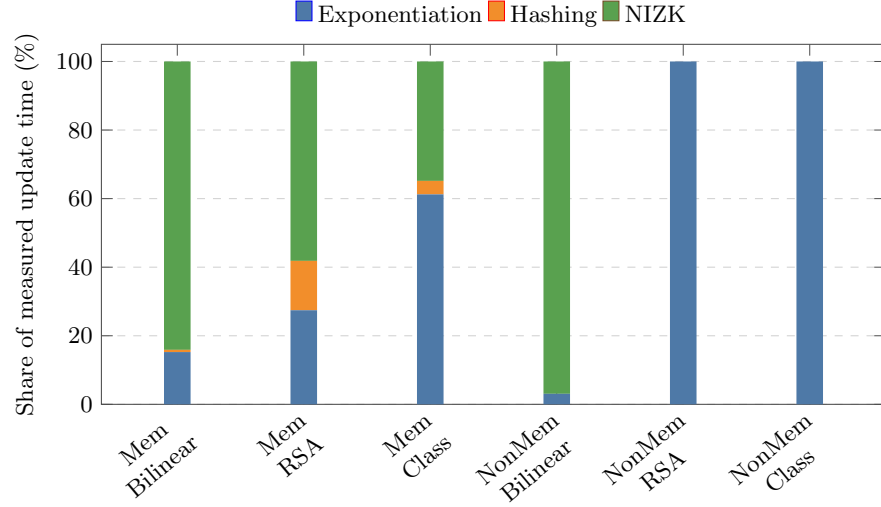


Fig. 15: Runtime breakdown in terms of exponentiation, hashing and NIZK proving of (non-)membership proof updates when updating for *a single element*.

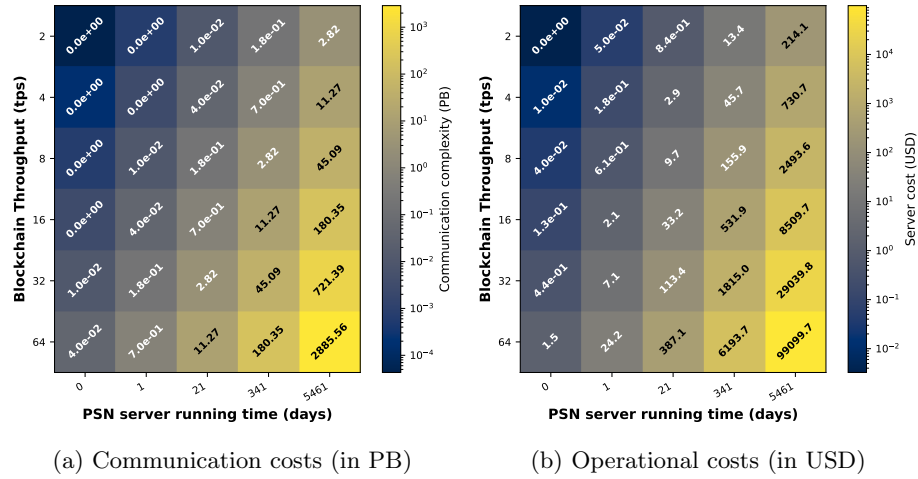


Fig. 16: Stateless Blockchain case study. Communication costs (in PB) and operational costs (in USD) of a PSN server for a given stateless blockchain throughput and uptime for the RSA accumulator.

RSA proof delegation algorithms incur higher communication and operational costs to the PSN server compared to the bilinear accumulator case (again, proof delegation running times for larger update sizes were approximated from Figure 9a using linear regression). We believe that the measured operational costs are reasonable for a real-world PSN deployment. For example, running a PSN server would cost $\approx 100\text{K USD}$ when updating RSA membership proofs for the users of a stateless blockchain with 64 transaction per second for an entire year. On the other hand, these numbers translate into $\approx 5.94 \times 10^{-10}$ USD cost for the client for each transaction submitted, which we can consider to be an economically negligible cost.